J ∫ Skripte

09.9.0426

Kryptologie ICS.KRYPTOG

Folien zur Präsenz 4

„Einweg- und Hashfunktionen sowie MAC's“, HS 26



Bild aus Klaus Schmeh „Kryptografie. Verfahren–Protokolle–Infrastrukturen“, dpunkt Verlag.

©Josef Schuler, dipl. math., dipl. Ing. NDS ETHZ, MSc Applied IT-Security, Feldhof 25, 6300 Zug, j.schuler@bluewin.ch resp. josef.schuler@hslu.ch

Inhaltsübersicht

- Einweg- und Hashfunktionen sowie MAC's
  - In Kap. 5.4 & 6.5 im JS Skript „Einführung in die Kryptologie“ wurden Hashfunktionen schon angesprochen, resp. ad hoc eingeführt. Dies im Zusammenhang mit der „Stellvertreterfunktion“ des Hashs bei der Signatur.
  - Nun besprechen wir die Kap. 14, 8.3, 27 & 8.4 im JS Skript „Einf. i. d. Kryptologie.“.
    - In Kap. 14 „Hashfunktionen“ gehen wir vom allg. Begriff „Einwegfunktion“ über zum Begriff „Hashfunktion“.
    - Kap. 8.3 „Integritätsschutzmechanismen“ beschreibt, wie man mit Blockchiffren (CBC-MAC) resp. mit Hashfunktionen (HMAC) sogenannte MAC's (Message Authentication Code) erzeugen kann, so dass die Unversehrtheit (= Integrität = Integrity) einer Meldung nachgewiesen werden kann.
    - In Kap. 27 „Einwegfunktionen“ beschreiben diesen Funktionstyp im Detail.
    - Kap. 8.4 „Randomgenerator“ beschreibt, wie man mit Blockchiffren spezifische Einwegfunktionen erzeugt.
  - Kap. 24 „Prüfzifferberechnung“ ist eine Ergänzung und ***nicht prüfungsrelevant!!***
  - ***Bemerkung:*** Mit „CBC-MAC“ sind im Folg. immer CBC-MAC ähnliche Algorithmen gemeint, also die Berechnung eines MAC's mit einem Blockchiffrierer.

Der Slogan zu dieser Präsenz

Eine kryptograph. sichere Hashfunktion $\overset{\Rightarrow}{\not\Leftarrow}$ ***ist eine Einwegfunktion.***

©Josef Schuler $J\int Skripte$

Lernziele

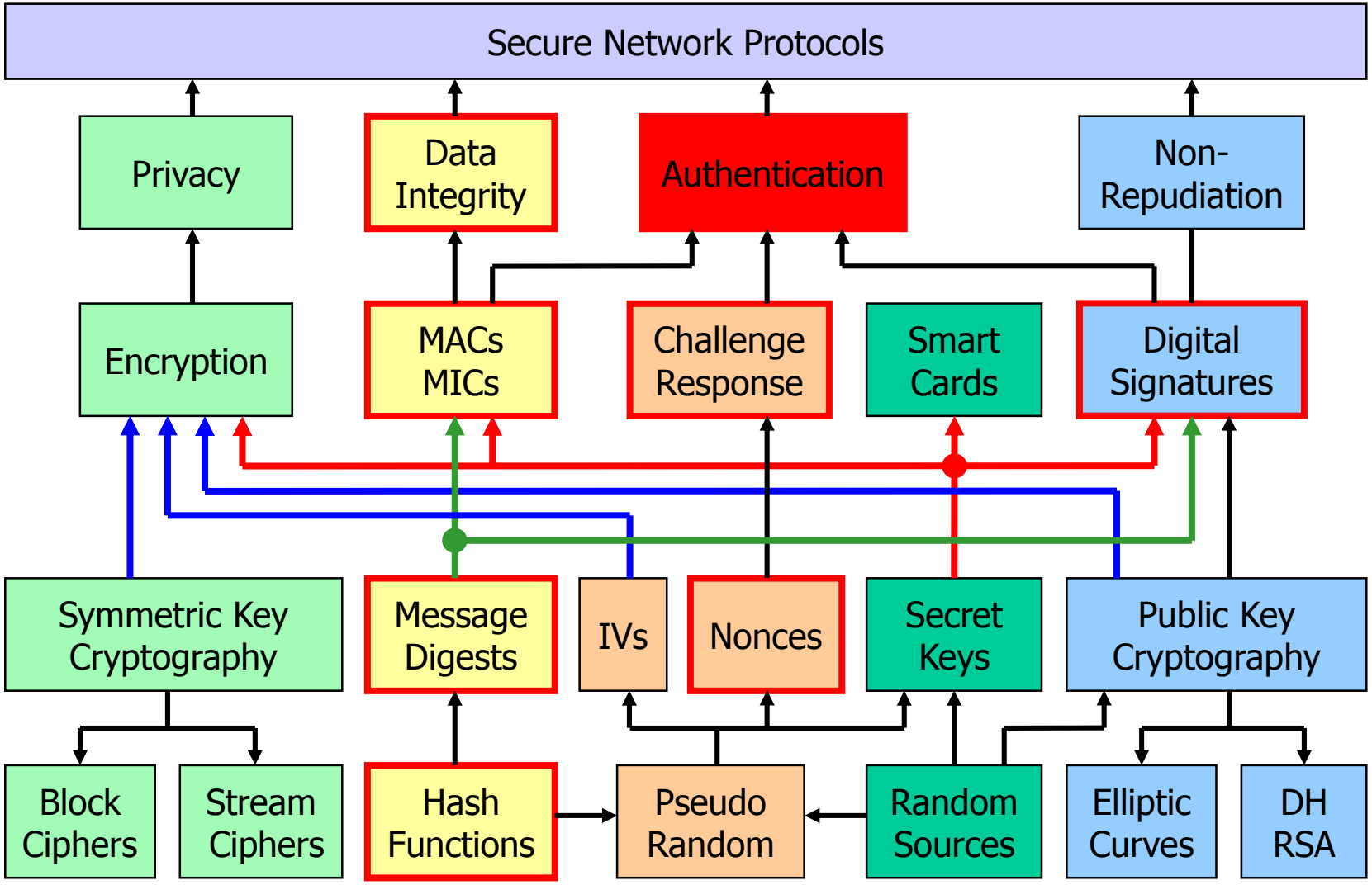
- Ich kenne die allgemeine Definition des Begriffs Einwegfunktion.
- Ich kenne die allgemeine Definition des Begriffs Hashfunktion.
- Ich weiss, was im Normalfall unter dem Begriff Hashfunktion (= MDC) verstanden wird.
- Ich kann die Begriffe Einwegfunktion, Hashfunktion, MDC & Prüfziffer richtig einordnen.
- Ich kenne die 3 Sicherheitseigenschaften einer Hashfunktion.
- Ich kenne den Pre-Image- und den Kollisionsangriff und deren Unterschiede.
- Ich kenne aktuelle Hashfunktionen und die weitere zentrale Eigenschaft.
- Ich kann den Unterschied zwischen einer keyed-Hashfunction und einer keyless-Hashfunction erklären.
- Ich kann den CBC-MAC (Grundfunktion und Variante nach ISO 9797-1) und HMAC aufzeichnen.
- Ich weiss, dass es viele Varianten von CBC-MAC ähnlichen Typen gibt. Ich kenne davon die Grundfunktion (ANSI X9.9) und die Variante nach ANSI X9.19, welche dem Algorithmus 3 (von 6) in ISO 9797-1 entspricht.
- Ich kann die Einsatzgebiete von MDC einerseits und CBC-MAC & HMAC andererseits benennen.
- Ich kenne die verschiedenen Typen von Einwegfunktionen.

- ***Wichtig:*** Die (#) bezeichn. Folien, Teile & Aufgaben sind ***nicht*** prüfungsrelevant!!

Verweise zur Literatur

- ***Für die Theorie:***
  - JS Skript „Einführung in die Kryptologie“, Kap. 8.3, 8.4, 14, 24 & 27.
  - ***Wichtig:*** Das Kap. 24 „Prüfzifferberechnung“ ist ***nicht*** prüfungsrelevant!
  - In [CP-D] die Kap. 6.1.2, 11 & 12.
- ***Aufgaben***
  - Aufgaben in den Folien & in den oben aufgeführten Kap. des JS Skript „Einf. in die Kryptologie“.
  - JS Skript „Aufgaben und Lösungen zum Modul KRYPTOG an der HSLU-I“, Kap. 2.2, „Aufgaben zur Präsenz 4, Einweg- und Hashfunktionen sowie MAC’s“.
- ***Bemerkungen:***
  - Die Kapitelnummerierung in den folgenden Folien entspricht derjenigen im oben erwähnten JS Skript „Einführung in die Kryptologie“. D.h. die Details zu den Folien können im Skript nachgelesen werden. Zudem hat es im Skript weitere Übungen und Beispiele.
  - ***Die Aufgaben- und Beispielnummerierung im JS Skript „Einführung in die Kryptologie“ und in den vorliegenden Folien stimmen nicht überein!***
  - ***Wichtig:*** Bitte beachten Sie, dass nur das Bearbeiten und Lernen der Folien nicht genügt. Das Durcharbeiten des oben erwähnten Kapitels in JS Skript „Einführung in die Kryptologie“ ist absolut zentral zum Bestehen der MEP.

Cryptographical Building Blocks v. Prof. Steffen



Allgemeine Einführung

- Die Übersichtszeichnung von Prof. A. Steffen …
  - … soll zeigen wie komplex die Kryptologie ist.
  - … zeigt einmal mehr, dass man in der Kryptologie nicht – wie z.B. in der Mathematik – „straight forward“ unterrichten kann. Man kommt nicht darum herum hie und da in Zyklen zu unterrichten, resp. zu lernen. D.h. gewisse Themen zu verschiedenen Zeitpunkten des Semesters und aus verschiedenen Sichtweisen zu betrachten. Hashfunktionen sind genau so ein Beispiel.
  - … ist in diesem Sinne nicht vollständig, es fehlen z.B. die „Einwegfunktionen“.
- Hashfunktionen:
  - Beim Diskutieren mit Security-Spezialisten verschiedener Couleur merke ich immer wieder, dass die Begriffe Hash, MDC, Signatur, HMAC usw. nicht immer exakt auseinander gehalten werden.
- Nicht alles ist Prüfungsstoff …
  - … die mit (#) bezeichneten Folien sind ***nicht*** Prüfungsstoff. Sie werden auch nur kurz angesprochen und nicht wirklich im Detail besprochen.
- ***Bemerkung***
  - Da wir schon in den bisherigen Präsenzen Hashfunktionen, MAC, HMAC usw. angesprochen haben, sind im Folgenden gewisse Folien telquel wiedergegeben.

Kap. 14

HASHFUNKTION

Kap. 14.1

EINLEITUNG

Eine übergeordnete Betrachtung

Betrachten wir unser Slogan zur Präsenz:

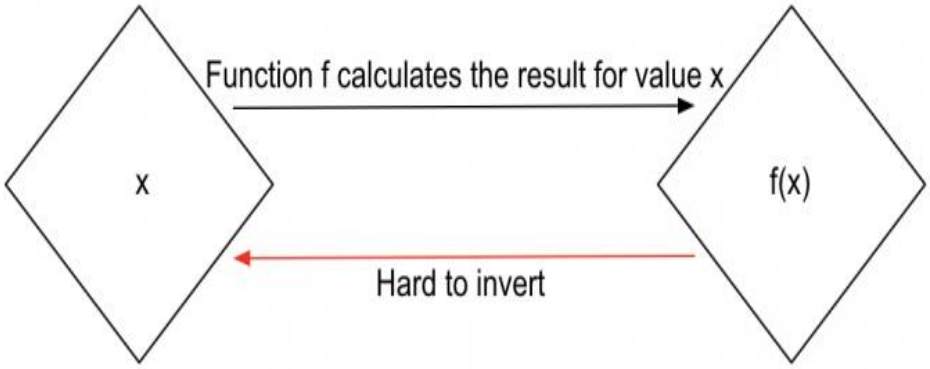
Eine kryptograph. sichere Hashfunktion $\Rightarrow$ $\not\Leftarrow$ ***ist eine Einwegfunktion.***

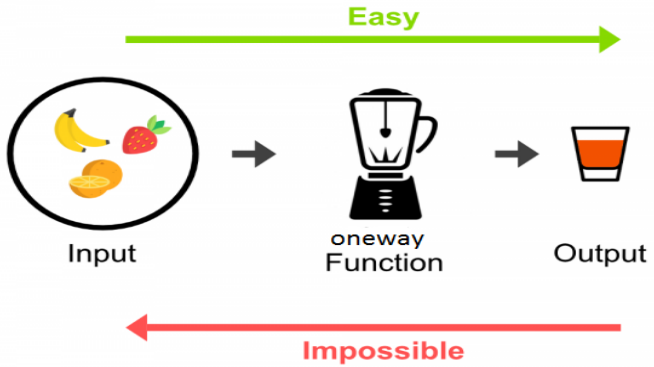
Somit ist eine Einwegfunktion etwas Allgemeineres als eine Hashfunktion.
Etwas salopp gesagt, eine Einwegfunktion lässt sich in eine Richtung „einfach“ berechnen, in die andere Richtung „unmöglich“ berechnen.
Sogar in [CP-D], S. 177 wird das nicht wesentlich anders definiert.

Definition 6.1 (Einwegfunktion)
Eine Funktion $f(\cdot)$ ist eine Einwegfunktion, wenn:

1. $y = f(x)$ rechentechnisch einfach und
2. $x = f^{-1}(y)$ technisch unmöglich zu berechnen ist.

Und so kommen wir zu zwei typischen Zeichnungen zu den Einwegfunktionen:



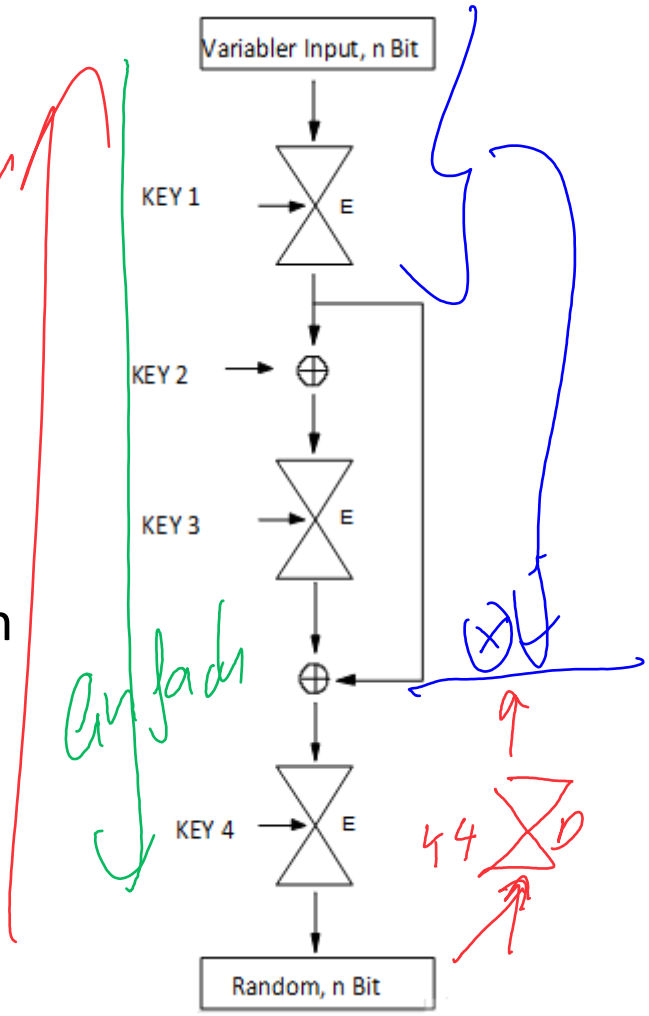


Und sofort kommt die erste Frage:

„Gibt es ein Gegenbeispiel? D.h. Gibt es ein Beispiel einer Einwegfunktion, die keine Hashfunktion ist?“

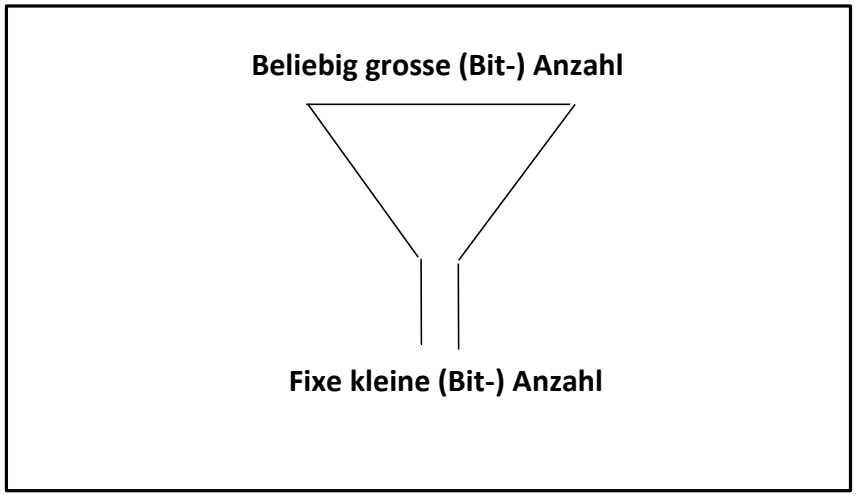
Die Antwort lautet: „Ja!“

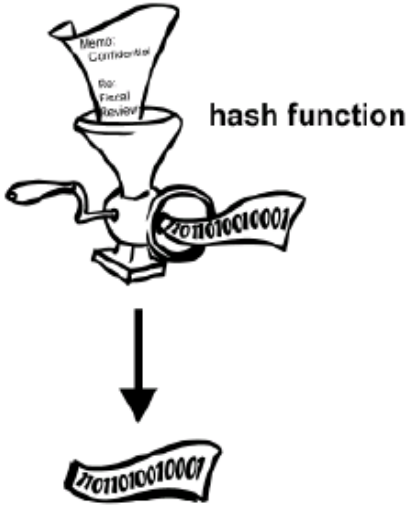
- In Kap. 8.4 „Randomfunktionen“ (siehe weiter hinten) wird eine Einwegfunktion, die keine Hashfunktion, besprochen.
- In der asymmetrischen Kryptographie beruhen z.B. der RSA und der Diffie-Hellman auf Einwegfunktion (mit oder ohne Trapdoor). Mehr dazu ebenfalls später in dieser Präsenz im Kap. 27.



Definition

Unter einer ***Hashfunktion*** verstehen wir eine Funktion, die die Elemente von einer „beliebig grossen“ Menge in eine „kleine fixe“ Menge abbildet. Wir können uns diese Funktion als Trichter vorstellen. Die kleine Bitzahl kann z.B. sein: 1, 16, 32, 64, 96, 128, 160, 224, 256, 384, 512.



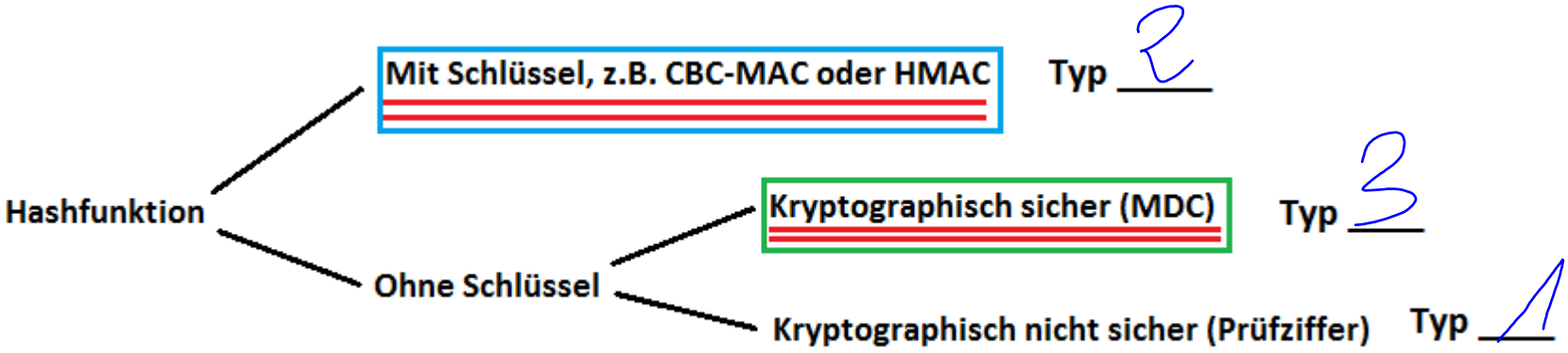


Wir kennen:

- Kryptographisch nicht sichere, schlüssellose Hashfunktionen = Berechnen von Prüfziffern.
- Kryptographisch sichere Hashfunktionen mit Schlüssel (= MAC)
- Kryptographisch sichere, schlüssellose Hashfunktionen (= MDC)

Typen von Hashfunktionen, Übersicht

Die unterstrichenen Funktionen sind kryptographisch sichere Hashfunktionen.



MAC = Message Authentication Code (er sollte besser MIC = Message Integrity Code heissen, da er Integrität und nicht die ganze Palette der Authentizität gewährt).

MDC = Manipulation Detection Code = „Hashfunktion“, wie sie üblicherweise verstanden werden. Eine Hauptanwendung ist die Stellvertreterfunktion bei der Signaturerstellung, resp. –verifikation.

Aufgabe 1

Betrachten Sie nun kurz – und nicht im Detail – die folgenden 4 Folien. Ordnen Sie Typ 1, 2 oder 3 in der Übersichtsgraphik zu.

Danach Aufgabe 3

1 5 4 5 0

Typ 1 von Hashfunkt. = Prüfziffern, Hausaufgabe

- Kryptographisch nicht sichere (schlüssellose) Hashfunktionen
  - Prüfzifferberechnung, letzte Ziffer der ISBN-Nummer = eine EAN Nummer (**EAN** = **E**uropäische **A**rtikel **N**ummerierung).

    Die Prüfziffer der EAN-Nummern (13. Ziffer) berechnet sich, indem man die ersten zwölf Ziffern abwechselnd mit 1 und 3 multipliziert (links mit 1 anfangen) und diese Produkte summiert. Die Prüfziffer ist die Differenz der Summe zum nächsten Vielfachen von 10. Falls die Summe durch 10 teilbar ist, ist die Prüfziffer die 0.

  - **Beispiel: 978381582086[?]**

$9 \cdot 1 + 7 \cdot 3 + 8 \cdot 1 + 3 \cdot 3 + 8 \cdot 1 + 1 \cdot 3 + 5 \cdot 1 + 8 \cdot 3 + 2 \cdot 1 + 0 \cdot 3 + 8 \cdot 1 + 6 \cdot 3$

$= 9 + 21 + 8 + 9 + 8 + 3 + 5 + 24 + 2 + 0 + 8 + 18 = 115$

$115 + \mathbf{5} = 120 \Rightarrow$ Prüfziffer: 5

  - Eine einstellige Prüfziffer {0; 1; …; 9}. Die Prüfziffer ermöglicht das Erkennen von Eingabe- und Lesefehlern: Erkannt werden ein Einzelfehler (genau eine Ziffer falsch) und die meisten Vertauschungen von zwei Nachbarziffern (ISBN-13: außer 0↔5, 1↔6, 2↔7, 3↔8, 4↔9), siehe
    https://de.wikipedia.org/wiki/Internationale_Standardbuchnummer#ISBN-13
    http://www.arndt-bruenner.de/mathe/scripts/pruefziffern.htm

Typ 1 von Hashfunktionen, Fort. Hausaufgabe

- Beispiel einer EAN Nummer mit Strichcode



- **Aufgabe 2**
  a) Überprüfen Sie die ISBN-Prüfziffer von [CP-D] 978-3-662-49296-3
  b) Überprüfen Sie die die EAN vom obigen Bild.
  c) Überlegen Sie sich, die oben erwähnten Eigenschaften.

- ***Bemerkung:*** Um einen reibungslosen Verlauf von der alten 10-stelligen ISBN Nummer zur neuen 13-stelligen EAN zu machen, hat man einfach ein neues „Land“ erfunden. 978 = das Land „Buch“.
- Für CH & Li gelten die Nummern 760 – 769
  Siehe https://de.wikipedia.org/wiki/GS1-L%C3%A4nderpr%C3%A4fix

- Weiterer Typus von Prüfziffer
  - CRC (Cyclic Redundancy Check) bei der physikalischen Übertragung.

- Wichtige Eigenschaft der Prüfziffer:
  - Ist ein Schutzmechanismus um unbewusste Veränderungen (Vertippen oder Bitfehler bei der Übertragung) einer Meldung zu entdecken.

Typ 2 von Hashfunktionen = MAC, HMAC

- Kryptographisch sichere Hashfunktionen mit Schlüssel (MAC)
  - CBC-MAC ähnliche Algorithmen, also MAC Berechnung mit einem Blockchiffrierer (cf. Kap. 8.3.1 & Kap. 8.3.2)
  - HMAC = MAC-Berechnung mit Hashfunktionen (cf. Kap. 8.3.3); das „H“ HMAC steht für „Hash“, also HMAC = Hash-MAC.
  - Ist ein Schutzmechanismus um bewusste Veränderungen einer Meldung zu entdecken. Dient zur Gewährung der Integrity und bietet automatisch auch Schutz vor Insertion.

- Die Hashfunktionen mit Schlüssel heissen MAC (Message Authentication Code) genannt. Eigentlich sollten sie MIC (Message Integrity Code) benannt werden. Letztlich ist das ein historisches Versäumnis, denn zu Beginn der 70‘er Jahre hat man (leider) Integrität mit Authentizität gleichgesetzt. Wir wissen aber, dass Integrität (Unversehrtheit) nur ein Teilaspekt der Authentizität einer Meldung ist.

Typ 3 von Hashfunktionen = MDC

- Kryptographisch sichere (schlüssellose) Hashfunktionen.
  - Auch MDC (Manipulation Detection Code) genannt.
  - In PGP auch (MD-5) Fingerprint genannt.
  - Das Resultat eines MDC wird oft kurz „Hash“ bezeichnet.
  - Ein MDC alleine gewährt keine Integrität.
- Im nachfolgenden Kap. 14.2 geht es um diese MDC.
  - Deren (vielfältigen) Einsatz, z.B. Stellvertreterfunktion bei Signaturen.
  - Empfehlungen zur Verwendung
  - Usw.
- Das Berechnen eines Hashwertes einer Meldung bewirkt noch keine Integrität der Meldung. Denn ein Angreifer kann die Meldung verändern und dazu den zugehörigen Hashwert berechnen.

Aufgabe 3

Kreuzen Sie den richtigen Typus an.

Hashfunktion	Kryptographisch sicher		Kryptographisch nicht sicher
	Mit Schlüssel	Ohne Schlüssel	
1) Alph. Registratur			
2) Ablage nach Eingangsdatum			
3) Parity-Bit (*)			
4) Byte Addition (mod 256) (*)			
5) CRC (*)			
6) CBC-MAC			
7) HMAC			
8) Konv. Prüfsumme (*)			
9) MDC = „Hashfunktion“			

(*) Kann bei Interesse in Kap. 24 „Anhang 2: Prüfzifferberechnung“ nachgelesen werden. Diese Prüfzifferberechnungen sind aber *nicht* Prüfungsstoff.

Kap. 14.2

MDC = Schlüssellose, kryptograph. sichere Hashfunktion

Typ 3

Aufgabe 4

In der Einleitung (cf. Kap. 14.1) haben wir 3 Typen von Hashfunktionen definiert. Um welchen Typus handelt es sich hier?

Abmachung, Grundanwendung & Nomenklatur

- Nachdem wir nun einen Einblick in den ganz allgemeinen Begriff „Einwegfunktion“ und dem ebenfalls allgemeinen Begriff „Hashfunktion“ erhalten haben, ist es nun Zeit eine Abmachung zu treffen.

Unter dem Begriff „Hashfunktion“ wollen wir nun eine kryptographisch sichere, schlüssellose Hashfunktion wie z.B. SHA-2 oder SHA-3 verstehen. Der – nicht sehr vertraute – Begriff „MDC“ werden wir in diesem Sinne nur noch selten verwenden.

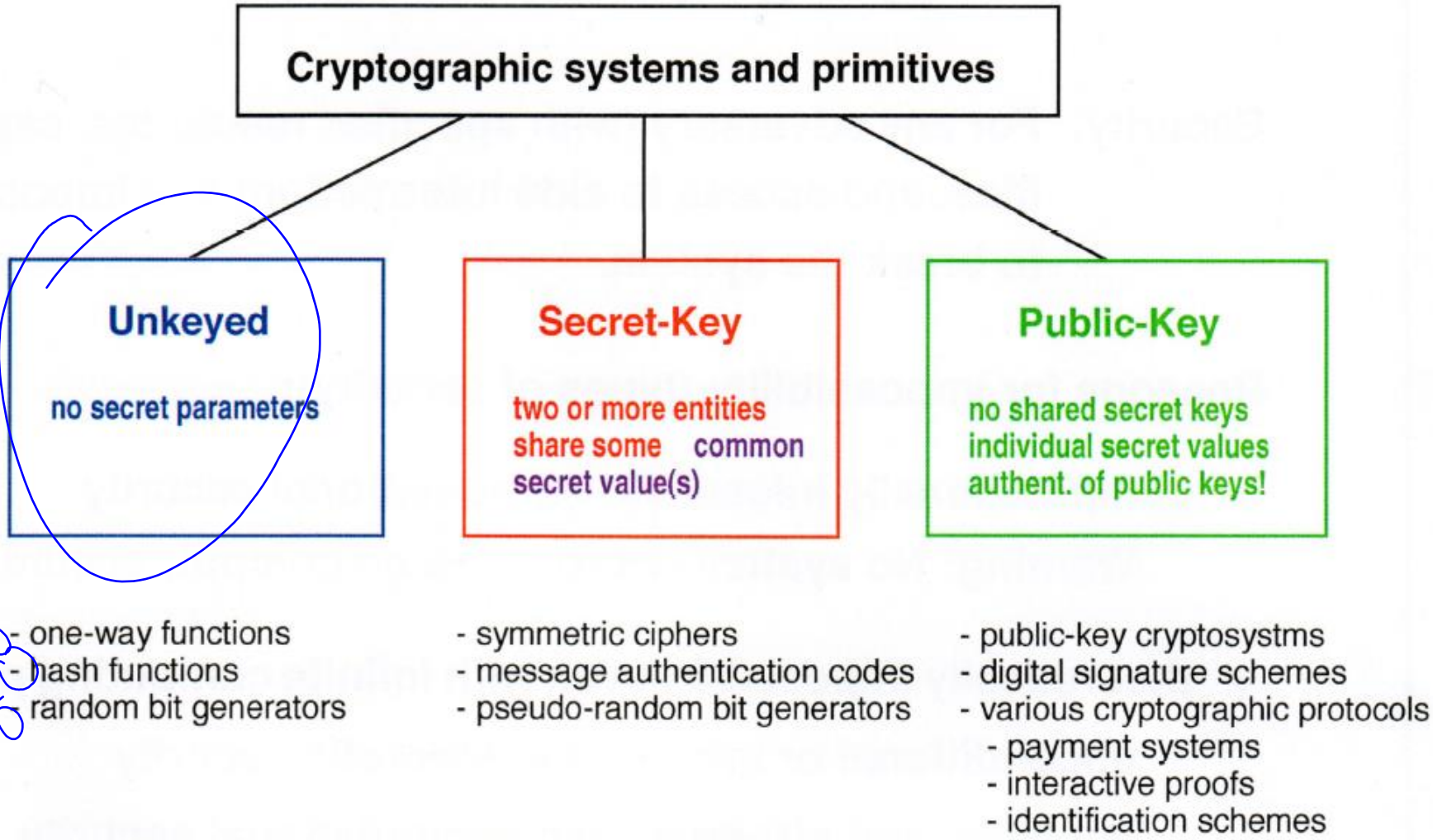
- Im Weiteren gilt:

Die primäre Grundmotivation, resp. –anwendung einer Hashfunktion ist die Stellverteterfunktion. D.h. ein (grosses) Dokument wird gehasht, damit danach der Hash – und damit das Dokument – signiert werden kann.

- Zur Nomenklatur: Wir schreiben …
  - … SHA-2, resp. SHA-3, wenn der Algorithmus allgemein gemeint ist.
  - … SHA2-256, SHA2-384 usw., resp. SHA3-256, SHA3-384 usw., wenn der Algorithmus und die Hashgrösse angegeben werden.

Einbettung von Hashfunktionen

In der Aufteilung der Kryptologie von Prof. Ueli Maurer (Kap. 2.1.3.1 im JS Skript, resp. Präsenz 0, Folie 11) beschreibt der dritte Block die „schlüssellose“ Kryptographie. Die Hashfunktion ist eine solche Ausprägung.



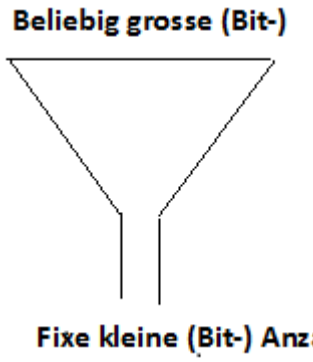
Sicherheitseigenschaften von Hashfunktionen

Aus den 2 Stichworten „schlüssellos“ und „Stellvertreterfunktion“ ergeben sich sofort verschiedene Fragen:

A) Gibt es überhaupt Sicherheitseigenschaften?

B) Was für Sicherheitseigenschaften soll so eine schlüssellose Stellvertreter-funktion haben?

Bis jetzt haben wir ja festgestellt, dass es hierzu sehr diffizile Betrachtungen und Angriffe gibt, auf die man nicht so ohne weiteres kommt.

<table>
<tr>
<td> 
</td>
<td>Die „beliebig grosse“ Bitanzahl kann mit $\{0,1\}^*$ beschrieben werden. Dabei charakterisiert „*“ das „beliebig“.

Die kleine, fixe Bitanzahl kann mit $\{0,1\}^n$ für $n \in \mathbb{N}$ beschrieben werden. Dabei charakterisiert „n“ die feste Anzahl Bit im Output, z.B. n = 256, 384 resp. 512 Bit.</td>
</tr>
<tr>
<td colspan="2">Somit ist also eine (schlüssellose) Hashfunktion eine Funktion
$h\colon \{0,1\}^* \to \{0,1\}^n$</td>
</tr>
</table>

Die 3 zentralen Sicherheitseigenschaften

1) **Urbildresistenz** (= Einwegeigenschaft = preimage resistance)

Für ein gegebenes $y \in \{0,1\}^n$ ist es praktisch nicht möglich einen Wert $x \in \{0,1\}^*$ zu finden mit $h(x) = y$.

2) **Schwache Kollisionsresistenz** (= zweite Urbildresistenz = 2nd preimage resistance = (weak) collision resistance)

Für ein gegebenes $x \in \{0,1\}^*$ und zugehöriges $h(x)$ ist es praktisch nicht möglich, einen Wert $x' \in \{0,1\}^*$ und $x' \neq x$ zu finden mit $h(x) = h(x')$.

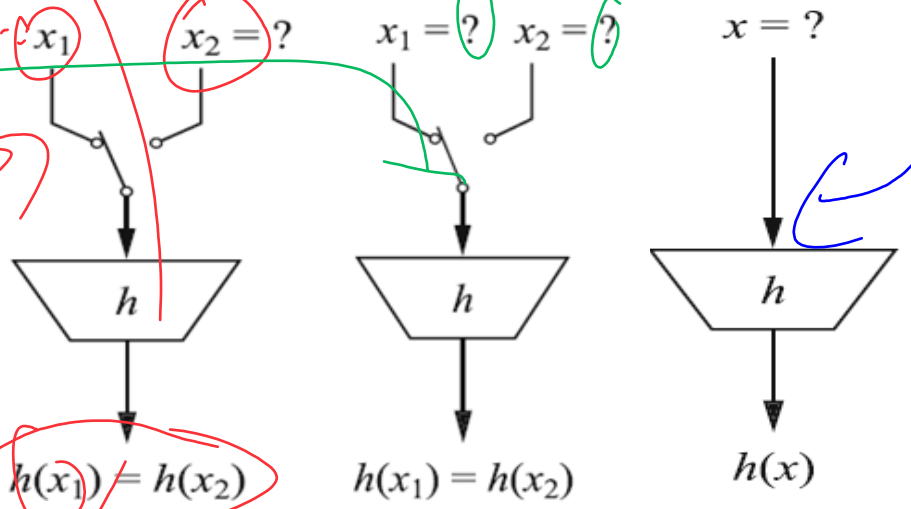
3) **Starke Kollisionsresistenz** (= Kollisionsresistenz = (strong) collision resistance)

Es ist praktisch nicht möglich zwei Werte $x, x' \in \{0,1\}^*$ und $x' \neq x$ zu finden, so dass $h(x) = h(x')$.

Aufgabe 5

a) Ordnen Sie die 3 Eigenschaften den Diagrammen richtig zu.

b) Erklären Sie nun, dass eine Prüfziffer keine kryptographisch sichere Hashfunktion ist.



Bemerkung:

Hashfunktionen müssen (natürlich) noch weitere (nicht sicherheitsrelevante) Eigenschaften erfüllen, z.B.

- **Deterministisch = Eigenschaft einer Funktion:** Die gleiche Eingabe liefert immer die gleiche Ausgabe.
- **Effizienz:** Die Berechnung des Hash-Werts muss schnell und effizient sein.

Die wichtigsten Hashfunktionen (MDC)

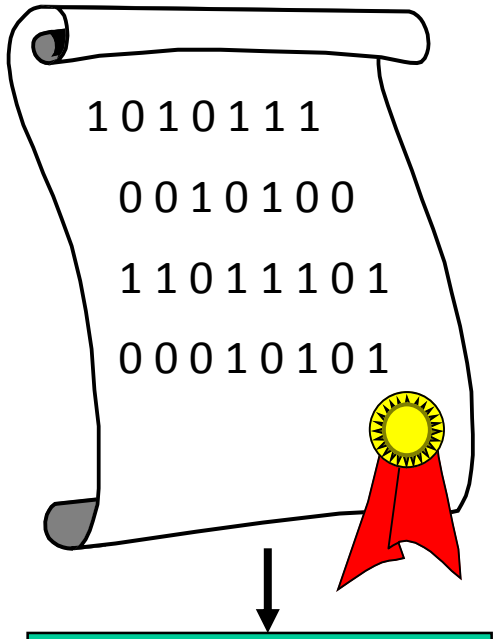
Namen	Output in Bit	Information zur Verwendung
MD-4	128	Darf nicht mehr verwendet werden!
MD-5	128	Darf nicht mehr verwendet werden! Wird in PRF (Pseudo Random Funktionen) aber noch verwendet.
SHA-0	128	Darf nicht mehr verwendet werden!
RIPEMD	160	Darf nicht mehr verwendet werden!
SHA-1	160	Darf für Signaturen nicht mehr verwendet werden! Als Grundfunktion für HMAC grundsätzlich OK. [BSI_TR02] empfiehlt aber sie auch dort nicht mehr einzusetzen. In Pseudo Random Function PRF wird sie vermutlich noch verwendet.
SHA2-224	224	Gemäss [BSI_TR02] wird in der tech. Richtlinie seit 2023 mind. 240 Bit (d.h. für Kollisionsangriffe mind. 120 Bit) gefordert. Daher seit 2023 für Signaturen nicht mehr zugelassen.
SHA3-224	224	Wie bei SHA2-224.
Seit 2023 sind nur noch die folg. Hashfunktionen für Signaturen zugelassen!		
SHA-2	256, 384 oder 512	Alle Einsätze OK; für Signaturen müssen seit 2023 mindestens 240 Bit genommen werden.
SHA-3	256, 384 oder 512	Wie bei SHA-2

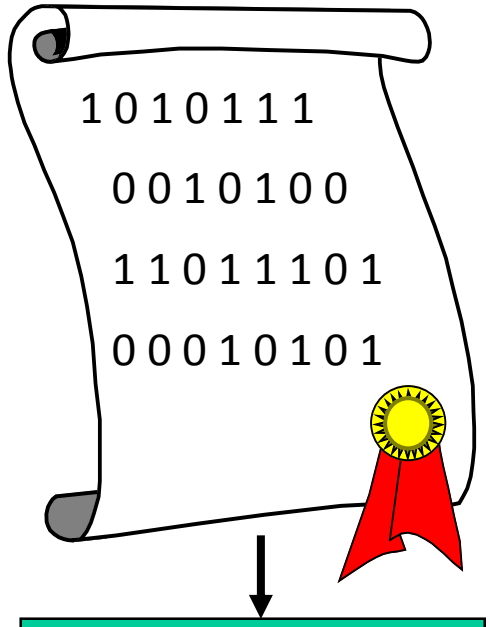
Wie sieht die Zukunft aus?

- Die SHA-2 Familie (genauer SHA2-256, SHA2-384 & SHA2-512), wird noch eine Weile eine erlaubte Alternative beim Einsatz in den Signaturen sein.
- Da sie aber im Wesentlichen auf SHA-1 beruht, erwartet man früher oder später entsprechende Attacken.
- Ein Wettbewerb – ähnlich wie bei der Ablösung des DES – wurde 2013 beendet.
- Der Sieger des Auswahlverfahrens ist der Hash mit Namen Keccak und heisst nun SHA-3. Er kann ebenfalls mit 256 bis 512 Bit eingesetzt werden, siehe vorherige Folie.
- D.h. zukünftige Entwicklungen wird es in absehbarer Zeit in diesem Bereich nicht mehr geben.

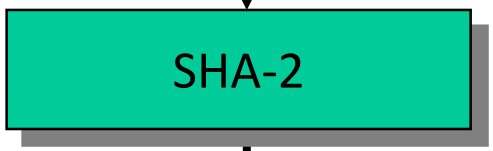
Schlüssellose kryptographisch sichere Hashfunktionen sind notwendige Hilfsmittel als Stellvertreterfunktion bei Signaturen

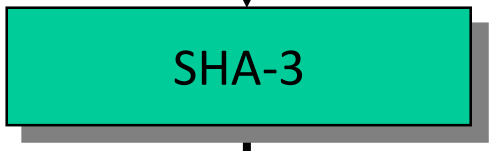
Dokument
oder
Meldung



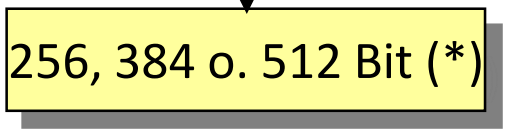


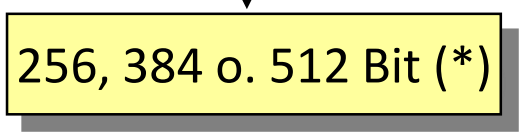
Hash Funktion





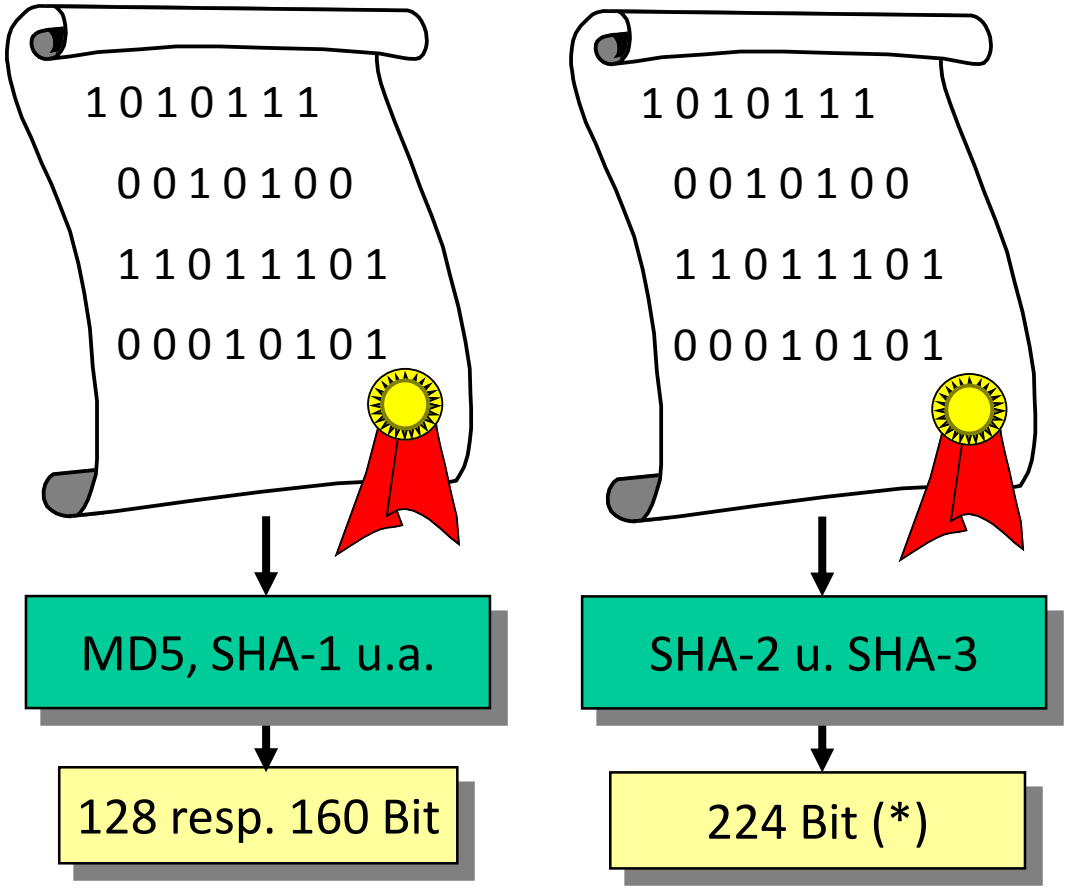
Message Digest MD
oder Hash
oder Fingerprint





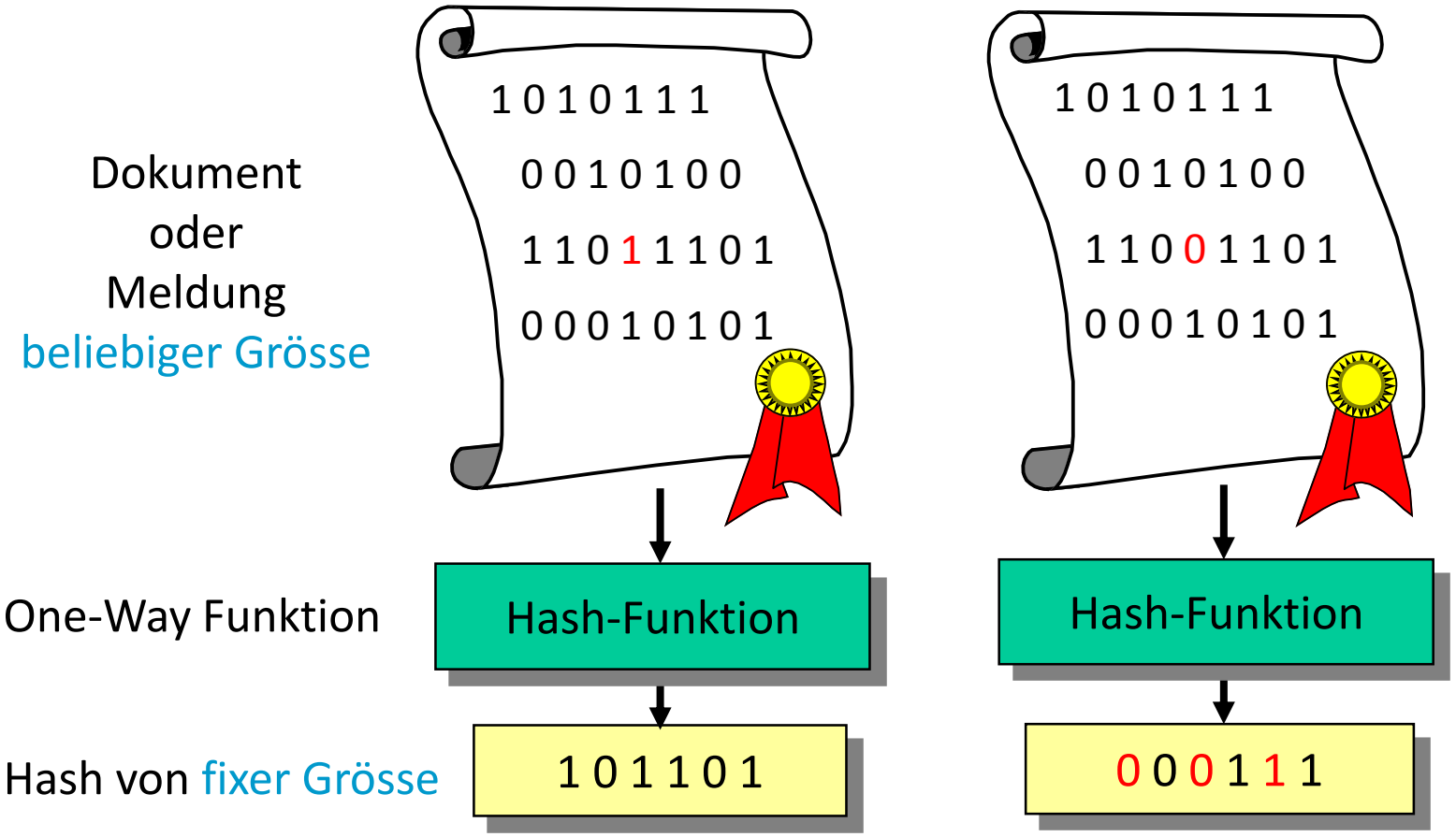
(*) Seit 2023 gilt mind. 240 Bit, also faktisch 256 Bit.

Diese schlüssellosen Hashfunktionen dürfen für Signaturen nicht mehr verwendet werden



(*) Gilt seit 2023

Weitere zentrale Eig. solcher Hashfunktionen



- Das Ändern eines einzigen Bits im Dokument hat zur Folge, dass im Hash jedes Bit mit Wahrscheinlichkeit ½ geändert wird.

Siehe Kap. 5.4.2 resp. Präsenz 1, wo die obige Eigenschaft mit „das Ändern eines einzigen Bits im Dokument hat zur Folge, dass im statistischen Mittel 50% der Bits im Hash geändert werden“ verglichen wird.

Einschub (Repetition): Binomialkoeffizient $\binom{n}{m}$

1) Es gilt: $\binom{n}{m} = \frac{n!}{m! \cdot (n-m)!} = \binom{n}{n-m}$ ***Beispiel:*** $\binom{11}{4} = \frac{11!}{4! \cdot 7!} = \binom{11}{7} = 330$

2) Bei $m = n/2$ ist der Binomialkoeffizient maximal, d.h. $\binom{n}{n/2}$ ist maximal.

3) $\binom{n}{m}$ beschreibt wie viele Möglichkeiten gibt es m Elemente auf k Plätze zu verteilen.
   Siehe dazu Lotto 6 aus 42 (Reihenfolge der Zahlen nicht berücksichtigt)
   - 42 Möglichkeiten für 1 Zahl.
   - $42 \cdot 41$ für 2 Zahlen mit Reihenfolge, $\frac{42 \cdot 41}{1 \cdot 2} = \binom{42}{2} = 861$ für 2 Zahlen o. RF.
   - …
   - $42 \cdot 41 \cdot \ldots \cdot 37$ für 6 Zahlen mit Reihenfolge, $\frac{42 \cdot 41 \cdot \ldots \cdot 37}{1 \cdot 2 \cdot \ldots \cdot 6} = \binom{42}{6} = 5'245'786$ für 6 Zahlen ohne Reihenfolge. → Beim CH-Lotto werden die Möglichkeiten wegen der Zusatzzahl noch mit 6 vergrössert, also $31'474'716$.

4) Analog nun für Ändern der Bits bei Hash der Grösse 256 Bit:
   - $\binom{256}{1} = 256$ Möglichkeiten für 1 Bit.
   - $\binom{256}{128} = 5{,}8 \cdot 10^{75}$ Möglichkeiten für die Hälfte, also 128 Bit.

Allg. Eig. v. krypt. sicheren Hashfunktionen, HA

1. Beim Ändern eines Inputbits, wird im statistischen Mittel die Hälfte des Hashs geändert, resp. genauer, wird jedes Bit des Hashes mit Wsk. ½ geändert.
2. Mit Hilfe von sogenannten Binomialkoeffizienten kann man zeigen, dass es am idealsten ist, wenn die Hälfte des Hashs geändert wird. Denn die Anzahl der Möglichkeiten sind maximal, resp. die Wahrscheinlichkeit zum Erraten minimal.
3. Allgemein ist $\binom{n}{m} = \frac{n!}{m!\cdot(n-m)!}$ bei $m = n/2$ maximal, d.h. $\binom{n}{n/2}$ ist maximal.
   - Siehe Blockchiffren: bei n = 128 → $2{,}4 \cdot 10^{37}$ → heute ungenügend!
   - Bei n = 160: $\binom{160}{80} = \frac{160!}{80!\cdot(160-80)!} = \frac{160!}{80!\cdot 80!} = 9{,}2 \cdot 10^{46}$ → ungenügend!
   - Bei n = 224: $\binom{224}{112} = \frac{224!}{112!\cdot(224-112)!} = \frac{224!}{112!\cdot 112!} = 1{,}4 \cdot 10^{60}$
   - Bei n = 256: $\binom{256}{128} = \frac{256!}{128!\cdot 128!} = 5{,}8 \cdot 10^{75} \rightarrow 10^{80}$ ≈ Anz. Atome im Weltall
   - Bei n = 384 → $1{,}6 \cdot 10^{114}$; bei n = 512 → $4{,}7 \cdot 10^{152}$
   - Es gibt total $\sum_{m=0}^{n}\binom{n}{m} = \binom{n}{0} = \binom{n}{1} + \cdots + \binom{n}{n} = 2^n$ mögliche Hashwerte:
     - $n = 384 \Rightarrow \sum_{m=0}^{384}\binom{384}{m} = \binom{384}{0} + \binom{384}{1} + \cdots + \binom{384}{384} = 2^{384} = 3{,}9 \cdot 10^{115}$
     - $n = 512 \Rightarrow \sum_{m=0}^{512}\binom{512}{m} = \binom{512}{0} + \binom{512}{1} + \cdots + \binom{512}{512} = 2^{512} = 1{,}34 \cdot 10^{154}$

Binomialkoeffizienten Resumée, cf. Präsenz 3

Bei den Hashfunktionen mit n = 384 oder n = 512 kann man grob Folgendes aussagen:

- Die mittleren 15% der Binomialkoeffizienten zusammengezählt ergeben > 99% der Gesamtsumme von 2^n.
- Die äusseren je 30% der Binomialkoeffizienten zusammengezählt ergeben nur einen winzigen Bruchteil von einem Promille der Gesamtsumme von 2^n.

$$\sum_{m=0}^{n} \binom{n}{m} = \underbrace{2^n}_{=100\%}$$

$$= \underbrace{\binom{n}{0} + \binom{n}{1} + \cdots +}_{<0{,}01\%} \ldots + \underbrace{\left(\frac{n}{2} - r\right) + \cdots + \binom{n}{\frac{n}{2}} + \cdots + \left(\frac{n}{2} + r\right)}_{>99\%} + \cdots \underbrace{+ \cdots + \binom{n}{n-1} + \binom{n}{n}}_{<0{,}01\%}$$

Was ja nichts anderes heisst als …

- … bei einer zufälligen Änderung des Hashes ändern sich zu 99,99…% die Hälfte +/- 7,5% der Bits. D.h. bei n = 384 Bit ändern sich zu über 99% zwischen 162 und 222 Bits. Bei n = 512 zu > 99% zwischen 220 und 292 Bits.
- … bei einer zufälligen Änderung des Hashes ändert sich nur in einem winzigen Bruchteil von einem Promille der Fälle weniger 30% der Bits oder mehr 70% der Bits. D.h. bei n = 384 Bit ändern sich sehr selten weniger als 115 oder mehr als 270 Bits. Bei n = 512 ändern sehr selten weniger als 155 oder mehr als 360 Bits.

Ein paar Zahlen für n = 512

Es gibt total 2^n mögliche Hashwerte: $n = 512 \Rightarrow 2^n = 1{,}34 \cdot 10^{154}$

- Anzahl genau die Hälfte geändert: $\binom{512}{256} = 4{,}7 \cdot 10^{152}$, das entspricht ca. 3,5% aller möglichen Hashwerten. D.h. in ca. 3,5% aller Fälle wird genau die Hälfte = 256 aller Bits geändert, resp. nicht geändert.
- Anzahl der geänderten Bits zw. 250 – 262: $\binom{512}{250} + \cdots + \binom{512}{262} = 5{,}8 \cdot 10^{153}$, das entspricht ca. 43,4% aller möglichen Hashwerten. D.h. in ca. 43,4% aller Fälle werden zwischen 250 – 262 Bits geändert, resp. 262 – 250 nicht geändert.
- Anzahl der geänderten Bits zw. 240 – 272: $\binom{512}{240} + \cdots + \binom{512}{272} = 1{,}1 \cdot 10^{154}$, das entspricht ca. 85,5% aller möglichen Hashwerten. D.h. in ca. 85,5% aller Fälle werden zwischen 240 – 272 Bits geändert, resp. 272 – 240 nicht geändert.
- Anzahl der geänderten Bits zw. 230 – 282: $\binom{512}{230} + \cdots + \binom{512}{282} = 1{,}315 \cdot 10^{154}$, das entspricht ca. 98,1% aller möglichen Hashwerten. D.h. in ca. 98,1% aller Fälle werden zwischen 230 – 282 Bits geändert, resp. 282 – 242 nicht geändert.
- Anzahl der geänderten Bits zw. 220 – 292 → ca. 99,88%.
- Anzahl der geänderten Bits zw. 200 – 312 → ca. 99,99995%.
- Anzahl der geänderten Bits < 100 oder > 412 → ca. $5 \cdot 10^{-44}$%.
- Anzahl der geänderten Bits < 50 oder > 462 → ca. 10^{-82}%.

Ein paar Zahlen für n = 256

Es gibt total 2^n mögliche Hashwerte: $n = 256 \Rightarrow 2^n = 1{,}15 \cdot 10^{77}$

- Anzahl genau die Hälfte geändert: $\binom{256}{128} = 5{,}8 \cdot 10^{75}$, das entspricht ca. 5% aller möglichen Hashwerten. D.h. in ca. 5% aller Fälle wird genau die Hälfte = 128 aller Bits geändert, resp. nicht geändert.
- Anzahl der geänderten Bits zw. 118 – 138: $\binom{256}{118} + \cdots + \binom{256}{138} = 9{,}4 \cdot 10^{76}$, das entspricht ca. 81% aller möglichen Hashwerten. D.h. in ca. 81% aller Fälle werden zwischen 118 – 138 Bits geändert, resp. 138 – 118 nicht geändert.
- Anzahl der geänderten Bits zw. 108 – 148 → ca. 99%.
- Anzahl der geänderten Bits zw. 98 – 158 → ca. 99,99%.
- Anzahl der geänderten Bits < 50 oder > 206 → ca. 10^{-21}%.

Kurzum:

Die Wahrscheinlichkeit, dass sich 20% oder weniger, resp. 80% oder mehr Bits im Hash ändern ist äusserst klein!!

Aufgabe 6 Ein paar Zahlen für n = 384

Sie dürfen auch das Maple File „Binomialkoeffizienten-Aufgabe 6“ in Ilias verwenden!

Es gibt total 2^n mögliche Hashwerte: $n = 384 \Rightarrow 2^n =$ _____

- Anzahl genau die Hälfte geändert: $\binom{\quad}{\quad} =$ _____, das entspricht ca. ___%

  aller möglichen Hashwerten. D.h. in ca. ____% aller Fälle wird genau die Hälfte = 192 aller Bits geändert, resp. nicht geändert.

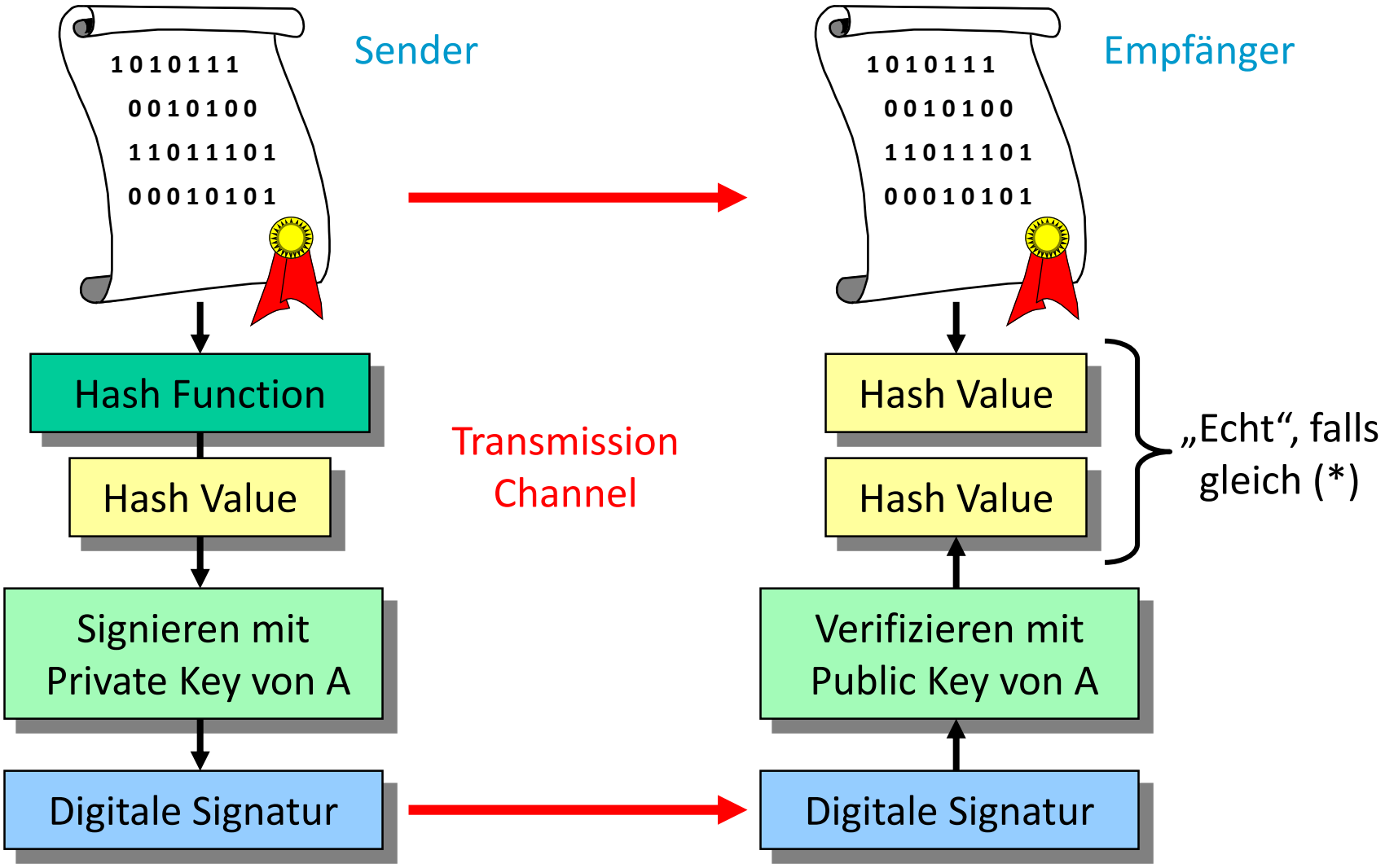
- Anzahl der geänderten Bits zw. 182 – 202: $\binom{\quad}{\quad} + \cdots + \binom{\quad}{\quad} =$ _____, das

  entspricht ca. _____% aller möglichen Hashwerten. D.h. in ca. _____% aller Fälle werden zwischen 182 – 202 Bits geändert, resp. 202 – 182 nicht geändert.

- Anzahl der geänderten Bits zw. 162 – 222 → ca. _____%.
- Anzahl der geänderten Bits zw. 142 – 242 → ca. _____%.
- Anzahl der geänderten Bits < 100 oder > 284 → ca. _____%.
- Anzahl der geänderten Bits < 50 oder > 334 → ca. _____%.

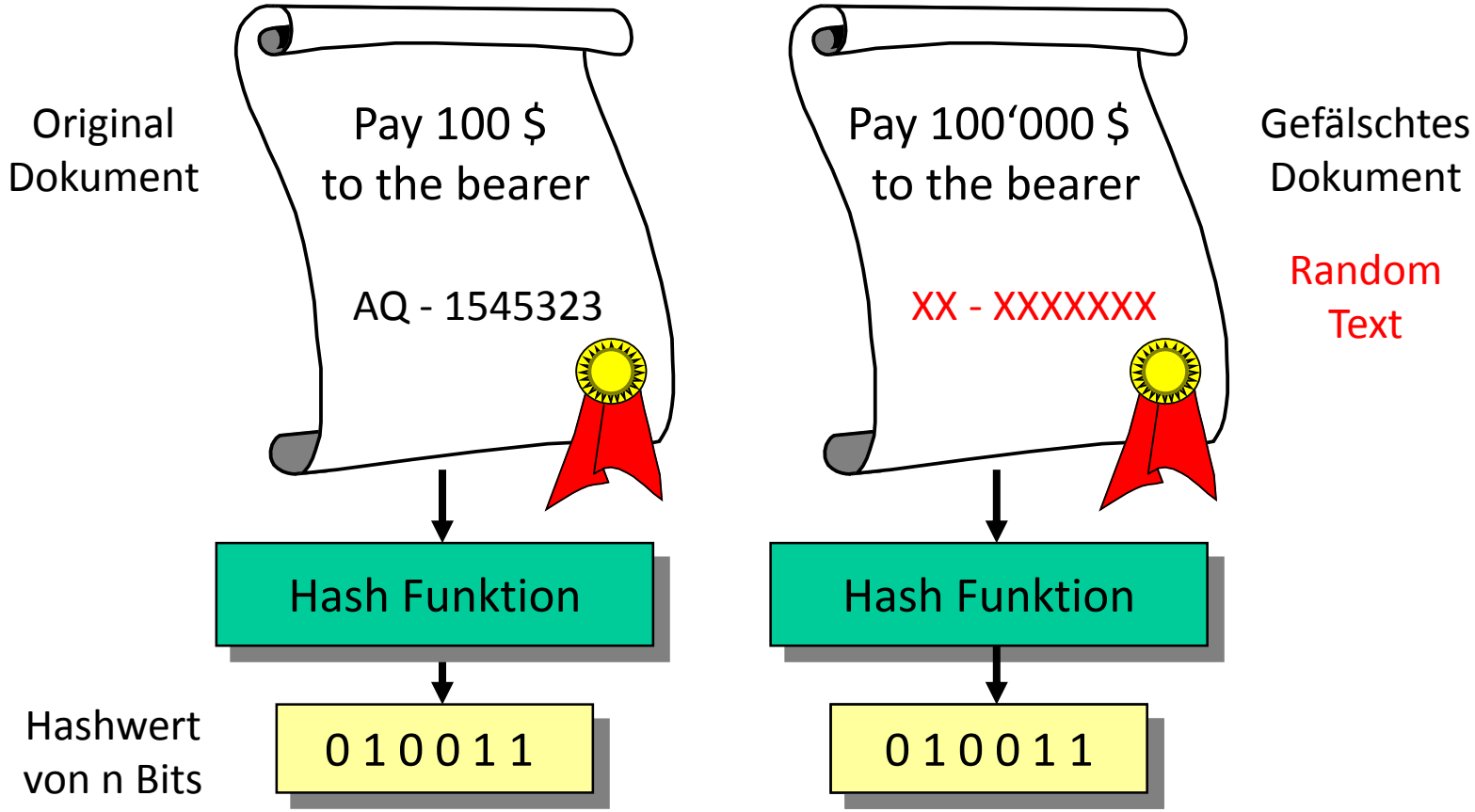
Die Rolle des MDC bei der digitalen Signatur

Public Key Krypto. 3: Digitale Signatur (z.B. RSA)



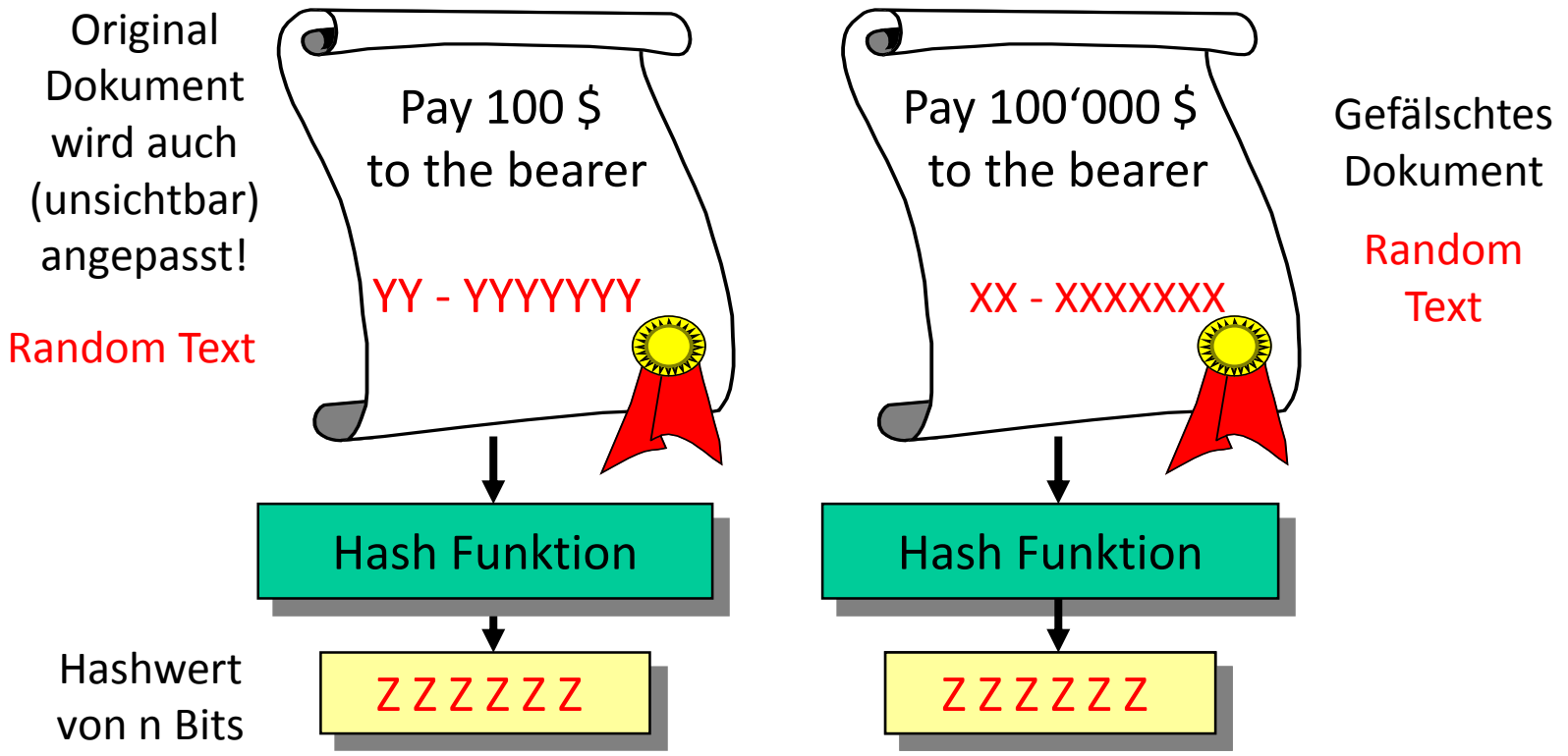
(**) zu 99,99… 9% echt falls gleich, aber zu 100% gefälscht, falls nicht gleich!

Die Gefahr 1 bei Signaturen = Pre-Image Angriff: Fälschen und Zurechtmachen eines Dokumentes



- Der Pre-Image Angriff ist ein Angriff auf die schwache Kollisionsresistenz.
- Dieser Angriff ist ***realistisch*** – da nur das gefälschte Dokument verändert werden muss – aber ***schwierig, sprich aufwändig***.
- Bei einem n-Bit Hash sind im stat. Mittel ca. 2^n Versuche nötig, um ein zweites Dokument mit gleichem Hash zu finden. In der Nomenklatur der $\mathcal{O}$ – Notation beträgt der Aufwand $\mathcal{O}(2^n)$.

Gefahr 2 bei Sig.: Kollisionsangriff (Geburtstags-Attacke) Originaldokument wird auch gefälscht!



- Der Kollisionsangriff ist ein Angriff auf die starke Kollisionsresistenz.
- Dieser Angriff ist eher ***unrealistisch*** – da das Original verändert werden muss – aber ***einfacher, sprich weniger aufwändig***, nämlich nur die Wurzel des Aufwandes des Pre-Image Angriffs.
- Bei einem n-Bit Hash sind im stat. Mittel nur die Wurzel, also $2^{\frac{n}{2}} = \sqrt{2^n}$ Versuche nötig, um zwei Dokumente mit gleichem Hash zu finden. In der Nomenklatur der $\mathcal{O}$ − Notation beträgt der Aufwand $\mathcal{O}\left(\sqrt{2^n}\right)$.

Sicherheitseigenschaften vs. Attacken

1) **Urbildresistenz** (oder Einwegeigenschaft)

Für ein gegebenes $y \in \{0,1\}^n$ ist es praktisch nicht möglich einen Wert $x \in \{0,1\}^*$ zu finden mit $h(x) = y$.

Diese Eigenschaft muss eine krypt. Hashfunktion per Definition erfüllen.

2) **Schwache Kollisionsresistenz** (= 2-te Urbildresistenz o. 2nd-Pre-Image Eig.)

Für ein gegebenes $x \in \{0,1\}^*$ ist es praktisch nicht möglich, einen Wert $x' \in \{0,1\}^*$ und $x' \neq x$ zu finden mit $h(x) = h(x')$.

Wenn eine krypt. Hashfunktion sicher gegen den Pre-Image Angriff ist, dann erfüllt sie die Eigenschaft der schwachen Kollisionsresistenz.

3) **Starke Kollisionsresistenz** (oder Kollisionsresistenz)

Es ist praktisch nicht möglich zwei Werte $x, x' \in \{0,1\}^*$ und $x' \neq x$ zu finden, so dass $h(x) = h(x')$.

Wenn eine krypt. Hashfunktion sicher gegen den Kollisionsangriff ist, dann erfüllt sie die Eigenschaft der starken Kollisionsresistenz.

Ziel: Eine Hashfunktion muss sicher gegen die starke Kollisionsresistenz sein.

Also: Sicher gegen den einfacheren, aber unrealistischen Angriff.

Bemerkungen und eine Grundsatzdiskussion

- Erfüllt eine krypt. Hashfunktion die Eigenschaft 3), dann erfüllt sie auch automatisch die Eigenschaft 2).
- Die starke Kollisionsresistenz ist „im Quadrat“ besser als die Schwache, dies wegen der Wahrscheinlichkeit $1/\left(2^{n/2}\right) = 2^{-n/2}$ versus $1/2^n = 2^{-n}$. Das tönt widersprüchlich, ist es aber nicht. Bei der starken Kollisionsresistenz die Wahrscheinlichkeit (im Quadrat) grösser eine Kollision zu finden als bei der schwachen. Darum muss eine krypt. Hashfunktion, die die starke Kollisionsresistenz erfüllt besser sein; z.B. n doppelt so gross.
- SHA-2 und SHA-3 erfüllen insbesondere die Eigenschaft 3). Dies taten übrigens auch SHA-1, RIPEMD & MD5, aber bei n = 128, resp. 160 Bit wird $\sqrt{2^n} = 2^{64}, resp.\, 2^{80}$ der Aufwand im Bereich des Möglichen → cf. drei Folien weiter hinten.

Die Grundsatzdiskussion (nach der vertieften Behandlung von dem CBC-MAC & HMAC)

In Präsenz 3 wurde der CBC-MAC kurz angeschaut, und er wird weiter hinten nochmals besprochen. Ebenso wird der HMAC noch im Detail betrachtet.
Nun stellt sich die Frage, ob die drei vorhin besprochenen Sicherheitseigenschaften auch für den CBC-MAC und HMAC gelten müssen.

Aufgabe 7

Nach dem Behandeln vom CBC-MAC und HMAC machen Sie sich (ev. zu Zweit oder zu Dritt) Gedanken zu dieser Thematik.

Das Geburtstagsparadoxon, Teil 1

- Wir nehmen an, die Geburtstage sind über das ganze Jahr gleichverteilt, und die Geburtstage von 2 Personen seien unabhängig voneinander, also z.B. keine Zwillinge.
- Wie gross ist die Wahrscheinlichkeit, dass eine zufällig angetroffene Person P am gleichen Tag Geburtstag hat wie Sie?
  - Klar $p = \frac{1}{365}$, resp. $\overline{p} = 1 - \frac{1}{365} = \frac{364}{365}$, dass P ***nicht*** am gleichen Tag Geb. hat wie Sie.

- Gleiches Gedankenspiel mit zwei zufällig angetroffenen Personen P_1, P_2
  - $\overline{p} = \left(1 - \frac{1}{365}\right)^2 = \left(\frac{364}{365}\right)^2$, dass P_1 & P_2 ***nicht*** am gleichen Tag Geb. haben wie Sie. D.h. $p = 1 - \overline{p} = 1 - \left(\frac{364}{365}\right)^2 \approx 0{,}0058$
- Gleiches Gedankenspiel mit n zufällig angetroffenen Personen
  - $\overline{p} = \left(1 - \frac{1}{365}\right)^n = \left(\frac{364}{365}\right)^n$, dass n Personen ***nicht*** am gleichen Tag Geb. wie Sie haben. D.h. $p = 1 - \overline{p} = 1 - \left(\frac{364}{365}\right)^n$

- **HA: Aufgabe 8** Wie viele Leute müssten nun mit Ihnen in einem Raum sein, damit die Wsk. grösser als 50% ist, so dass jemand am gleichen Tag Geburtstag hat wie Sie?
- Benutzen Sie zur Kontrolle das Maple File „Geburtstagsparadoxon“ in Ilias.

Das Geburtstagsparadoxon, Teil 2

- Nun betrachten wir das Taubenschlagprinzip von Dirichlet (siehe DMATH). Wie viele Personen müssen in einem Raum sein, damit zu 100% zwei (beliebige) Personen am gleichen Tag Geburtstag haben?
  - Klar, wenn 366 (resp. 367, wenn wir Schaltjahre auch berücksichtigen) Personen im gleichen Raum sind, so muss es zwangsläufig zwei Personen geben, die am gleichen Tag Geburtstag haben → cf. Taubenschlagprinzip.
- Wie viele Leute müssten nun in einem Raum sein, damit die Wsk. grösser als 50% ist, so dass sich im Raum zwei Personen befinden, die am gleichen Tag Geburtstag haben?
- Diese Aufgabe (resp. die Herleitung der Lösung) ist nun viel schwieriger. In DMATH haben wir die Lösung einmal angetönt, in [CP-D], Kap. 11.2.3 ist die Herleitung im Detail beschrieben (nicht Lehr- und Prüfungsstoff). Kurz zusammengefasst gilt:
  - $\overline{p} = p(keine\ Kollision\ bei\ n\ Personen) = \left(1 - \frac{1}{365}\right) \cdot \left(1 - \frac{2}{365}\right) \cdot \cdots \cdot \left(1 - \frac{n-1}{365}\right)$
  - Bei n = 23 ist diese Wahrscheinlichkeit bei ca. 50%, d.h. ab 23 Personen wird die Wahrscheinlichkeit über 50%, dass es zwei Personen im Raum gibt, die am gleichen Tag Geburtstag haben.
  - Bei n = 40 ist diese Wahrscheinlichkeit bei ca. 10%, d.h. zu 90% gibt es zwei Personen im Raum, die am gleichen Tag Geburtstag.
  - Verallgemeinerung: ab ca. $\sqrt{n}$ Elementen ist die Wahrscheinlichkeit einer Kollision > 50%.
  - Mit dem Maple File „Geburtstagsparadoxon“ können Sie die Werte kontrollieren.

Mitte 2004: MD5 Kollision gefunden in 1 Std. (#)

d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325 7 1415a085125e8f7cdc99fd91dbd f 280373c5b
960b1dd1dc417b9ce4d897f45a6555d535739a c 7f0ebfd0c3029f166d109b18f
75277f7930d55ceb22e8adba79 c c155ced74cbdd5fc5d36db19b0a d 835cca7e3

d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325 f 1415a085125e8f7cdc99fd91dbd 7 280373c5b
960b1dd1dc417b9ce4d897f45a6555d535739a 4 7f0ebfd0c3029f166d109b18f
75277f7930d55ceb22e8adba79 4 c155ced74cbdd5fc5d36db19b0a 5 835cca7e3

These two 1024-bit values only differ in 6 bit positions

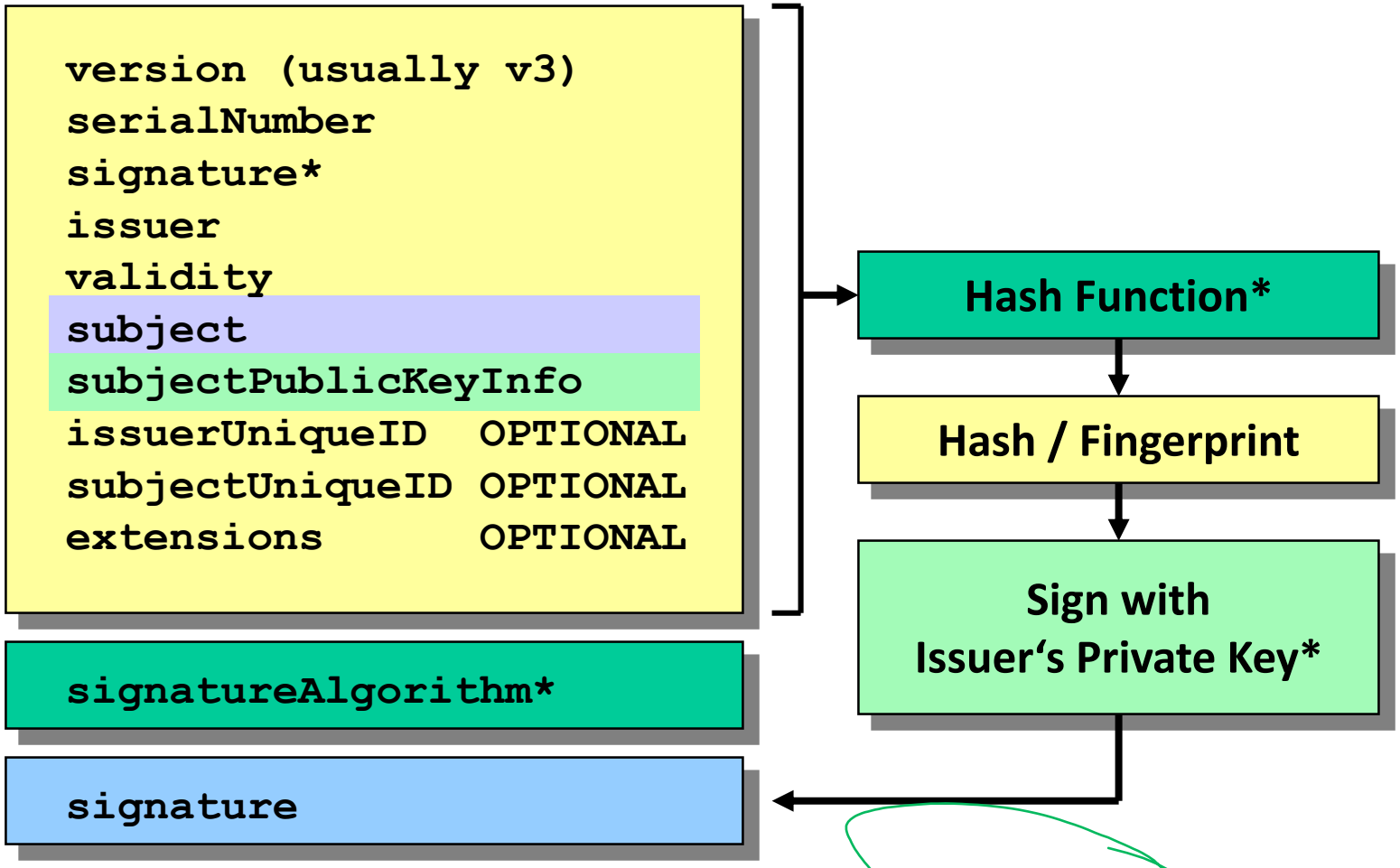
Common MD5 digest: **a4c0 d35c 95a6 3a80 5915 367d cfe6 b751**

- Xiaoyun Wang, Dengguo Feng, Xuejia Lai, and Hongbo Yu invented an algorithm that finds MD5 collisions on an IBM P690 computer within little more than one hour.
- Similar attacks exist for RIPEMD-128 and SHA-0.
- How large is the security margin of SHA-1?

Feb. 2005 & Juni 2009: Erfolgreiche Angriffe auf den SHA-1 (#)

- Pre-Image Angriff: Aufwand 2^{160} → unmöglich
- Kollisionsangriff: Aufwand 2^{80} → „möglich“
- Ist der Angriff mit deutlich weniger Versuchen als beim Brute-Force-Ansatz, gilt das Verfahren als geknackt.
- Feb. 2005 gelang das einer Chinesischen Forschergruppe: Sie haben ein Verfahren entwickelt, eine Kollision statt mit 2^{80} bereits mit 2^{69} Operationen zu ermitteln. Das reduziert die Zahl der notwendigen Operationen um den Faktor 2’048 = 2^{11}.
- Juni 2009 eine weitere Reduktion auf 2^{52} Operationen, also um den Faktor 131’072 = 2^{17}, also der Aufwand ist deutlich besser als $\mathcal{O}\left(\sqrt{2^n}\right)$.
- Wichtig ist, dass das immer Kollisionsangriffe waren (Birthday attacks auf die Kollisionsresistenz), Kollisionsangriffe sind in der Regel nicht für Betrüge ausnutzbar!
- Aktuell findet man Kollisionen ca. mit einem Aufwand von € 10’000.- (siehe [BSI_TR02], Kap. 4).
- Keine Panik: Um Signaturen zu knacken braucht es Pre-Image Attacken.
- ***Kein Problem mehr → Wir haben nun den SHA-3***

MDC in X.509 Zertifikat eingebettet, mehr dazu bei Prof. Portmann in der Präsenz zu PKI



* specifies algorithm used to sign certificate, e.g. SHA2-256RSA

Kap. 8.3, 14.3 & 14.4

CBC-MAC (*) und HMAC

(*) Mit CBC-MAC sind im Folgenden immer CBC-MAC ähnliche Algorithmen gemeint. Also die Berechnung eines MAC's mit einem Blockchiffrierer.

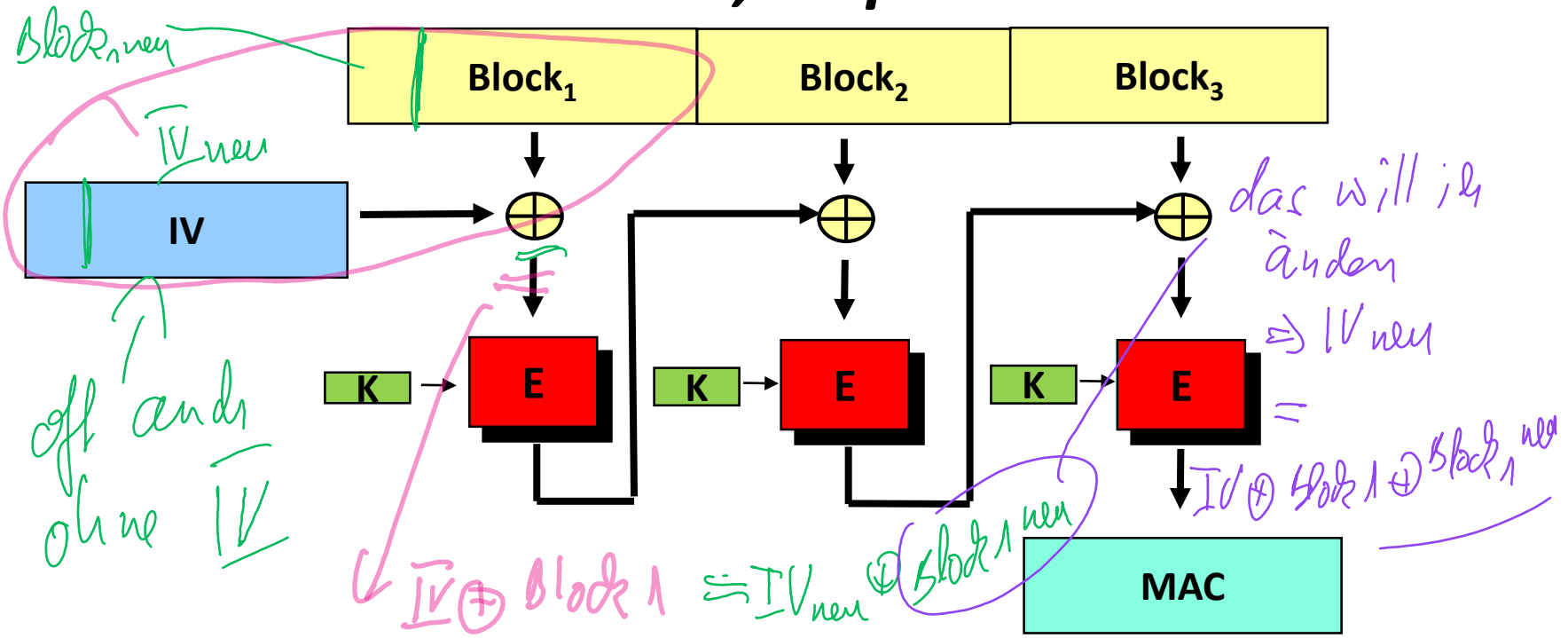
Wichtig:

Einige Folien sind aus Präsenz 2 – 3 eins zu eins übernommen.

Aufgabe 9

In der Einleitung (cf. Kap. 14.1) haben wir 3 Typen von Hashfunktionen definiert. Um welchen Typus handelt es sich hier?

Der CBC-MAC ANSI X9.9, resp. ISO 8730



- Der CBC-MAC, wie oben dargestellt, ist die ***Mutter*** aller Integritätsschutzmechanismen mit Blockchiffren. Er wird in allen Lehrbüchern dargestellt.
- Er hat gewisse (theoretische) Schwächen und ist eigentlich als Standard schon länger zurückgezogen.
- Es gibt diverse Anpassungen (cf. nächste Folie), gemäss ISO 9797-1 → ist aber auch als Standard zurückgezogen.
- In der Industrie gibt es viele weitere, individuell leicht angepasste Versionen.
- All diese Formen werden aber nach wie vor konkret verwendet. Dies gemäss Aussagen von Securityspezialisten, die „nahe am Geschehen sind“, so im Sinne „never change a winning team“.
- Die aktuellen MAC-Algorithmen haben sinngemäss eine analoge Form.

Der CBC-MAC: Eine Betrachtung des IV

Grundsätzlich hätte der IV die gleiche Bedeutung wie beim CBC-Verschlüsselung Modus, d.h.

a) Ohne IV müsste die SW checken, ob ein Block der zu MACen ist, der erste ist oder nicht.

b) Die gleiche Meldung mit gleichen Key K nochmals geMaced ergibt ohne IV den gleichen MAC.

Da der IV – wie beim CBC Verschlüsselungsmodus – in klar im Header mitgeschickt wird, hat das aber beim MAC einen gewichtigen Nachteil. Es gibt nun einen Angriff:

- Eve könnte nun einen neuen Block 1neu gerieren und einfügen, ohne das Resultat des ersten XOR ändert. Dann hat sie ohne Aufwand eine zweite Meldung mit gleichem MAC erzeugt.
- Eve muss demnach die folgende Berechnung durchführen:

$$IVneu \oplus Block1neu = IV \oplus Block1 \Rightarrow IVneu = Block1neu \oplus IV \oplus Block1$$

Konsequenz und Lösungsansatz:

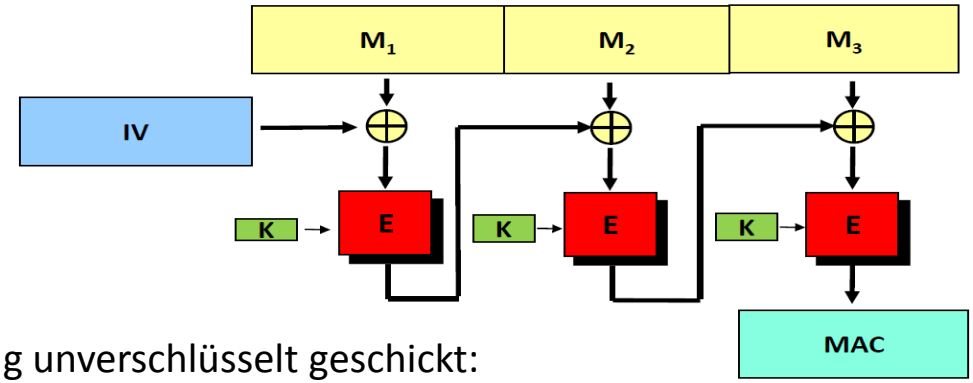
- Den IV einfach auf 00 … 00 setzen und nicht übertragen; Vorteil a) wird beibehalten.
- Vorteil a) beibehalten kann man auch indem ein (anderer) fixer IV definiert und ebenfalls nicht übertragen wird.
- Es gäbe aber auch die Möglichkeit – um damit auch den Vorteil b) beizubehalten – dass immer ein anderer IV erzeugt wird, dieser wie beim CBC-Verschlüsselungsmodus in klar mitschickt, aber der IV in die MAC-Berechnung (z.B. am Schluss) als Input eingeht.
- Das wird aber kaum so umgesetzt.

Bemerkung:

Man sieht daher in vielen Beschreibungen des CBC-MAC, dass er ohne IV da steht. Ob dann aus softwaretechnischen Gründen ein fixer 00 … 00 als IV verwendet wird, ist dann nebensächlich.

Schwäche beim CBC-MAC, Aufgabe 10

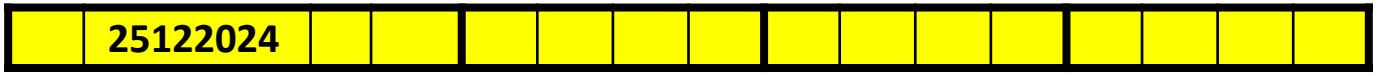
Eine Meldung wird mit dem CBC-MAC ausgestattet. Die Meldung wird unverschlüsselt und mit dem MAC versehen übermittelt. Im Folgenden sind alle Werte in HEX dargestellt, und Sie dürfen in HEX rechnen.



Folgende Meldung wird über die Leitung unverschlüsselt geschickt:



Dabei steht in Block M_1 im zweiten Feld ein Datum:



bis 17405

Und im IV stehen an derselben Stelle die folgenden Werte:



Der Inhalt der Meldungen M_2 und M_3 sind im Folgenden unwichtig und werden nicht angegeben. Eve kennt den Angriff auf den CBC-MAC mit dem Initial Value IV, sie fängt die ganze Nachricht ab und will das Datum 25. Dez. 2024 (= 25122024) auf den 21. Dez. 2024 ändern. Der MAC soll dabei ***unverändert*** bleiben.

Wie geht Eve vor?

Der CBC-MAC: Eine weitere Schwäche

Eine weitere Angriffsmöglichkeit beim CBC-MAC gibt es, wenn die Meldung kein Vielfaches der Blocklänge der Blockchiffre ist. In diesem Falle muss die Meldung gepaddet werden. Zunächst wurde einfach mit 00 … 00 gepadded.

Problem 1:

Ist das letzte Feld z.B. ein Geldbetrag, dann weiss man nicht wie gross nun der echte Betrag ist.

Lösungsansatz 1:

Das Padding wurde nun mit 10 … 00 gemacht. Somit wurde klar abgegrenzt, wo nun das Padding beginnt.

(Folge-)Problem 2:

Wenn die Meldung ein Vielfaches der Blocklänge des Blockchiffrieres ist, dann muss nun zusätzlich ein ganzer Paddingblock angehängt werden.

Lösungsansatz 2 (wurde leider nie standardisiert, in unseren Projekten aber implementiert):

Im Header (der Header geht auch in die MAC-Berechnung ein) der Meldung wird in einem Feld die Länge der Meldung, die in die MAC-Berechnung eingeht eingetragen.

Damit werden die obigen Angriffe unterbunden. Damit ist natürlich nicht gesagt, dass damit alle möglichen Angriffe, resp. Schwächen unterbunden werden.

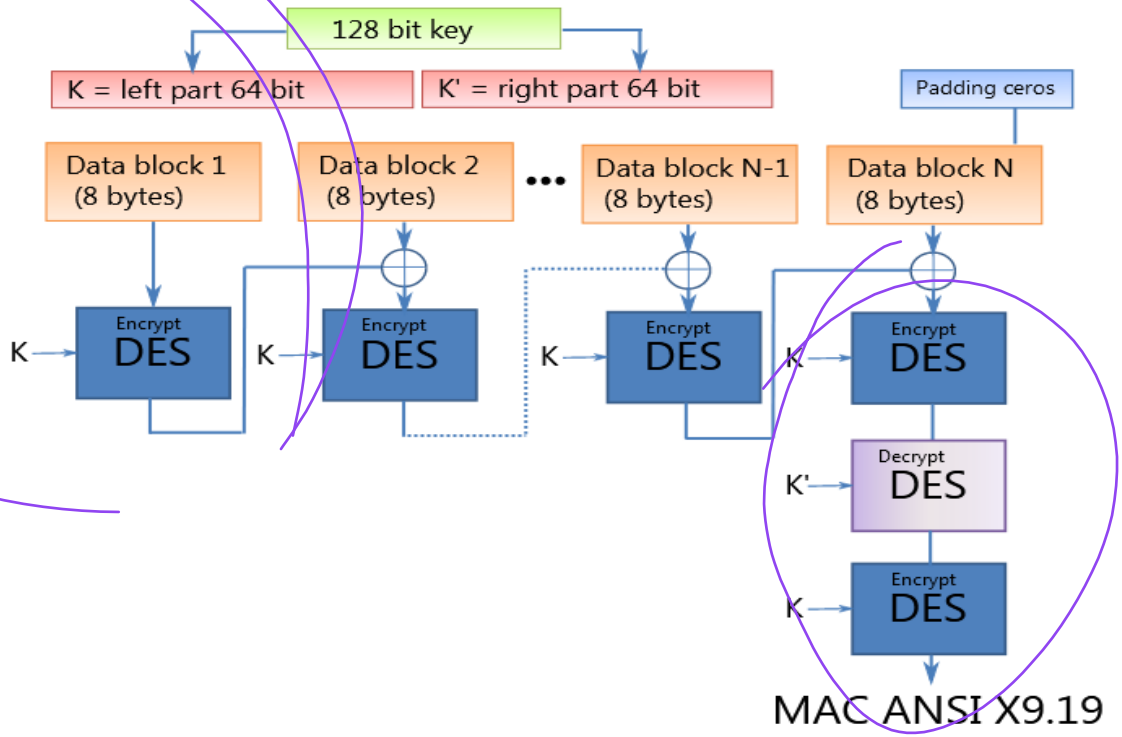
Problem 3:

Auch die nachfolgenden Verbesserungen des CBC-MAC haben die oben erwähnten Schwächen. Die Verbesserungen haben diesbezüglich keine Wirkung.

Lösungsansatz 3:

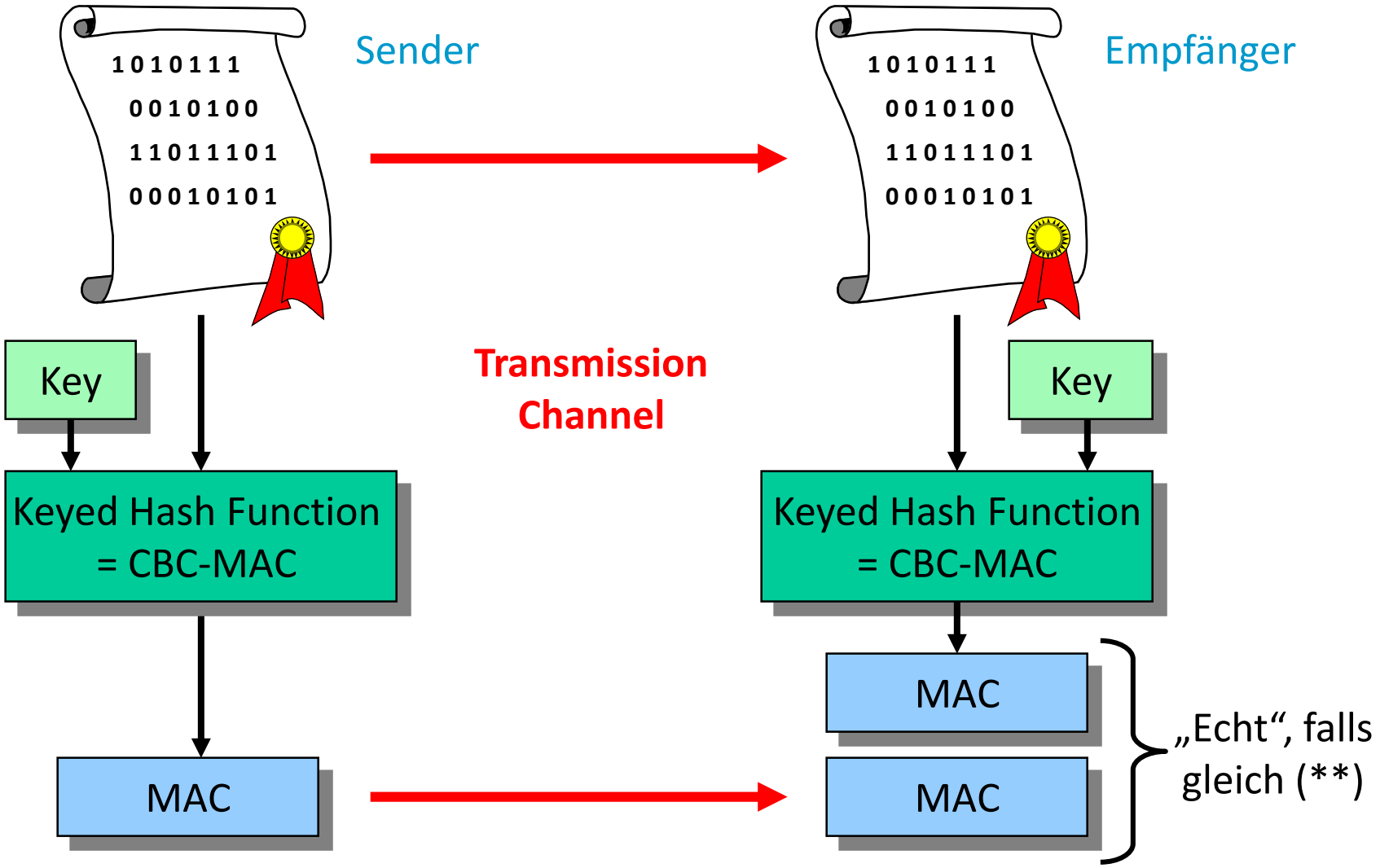
Definition von neueren MAC-Algorithmen wie CMAC usw.

Der CBC-MAC nach ISO 9797-1, resp. ISO 9807



- In ISO 9797-1 wurden 6 Varianten definiert, die oben dargestellte ist eine dieser Varianten und wurde als ANSI X9.19 Standard aufgeführt.
- Es gibt auch neuere Varianten, eben vor allem auch im Zusammenhang von kombinierten Modi.
- ***Bemerkung:*** Anstatt DES könnte natürlich auch AES oder ein anderer Blockchiffrierer stehen. Die Schlüsselgrössen müssten dann entsprechend angepasst werden. Im grünen Block würde bei AES als Blockchiffre 256, 384 resp. 512 bit key stehen.
- ***Bitte beachten Sie:*** Der DES hat nur eine Schlüsselgrösse von 56 Bit, und damit haben 2 DES Schlüssel nur 112 Bit. Aber solche Fehlangaben gibt es häufig bei Graphiken aus dem Internet!!

Verifikation mit CBC-MAC (*)



(**) zu 99,99… 9% echt falls gleich, aber zu 100% gefälscht, falls nicht gleich!

Spezielle Modi (#)

- Mit dem GCM (Galois Counter Mode) wurde ein Mode kreiert, der sowohl verschlüsseln wie MAC'en kann → sie Kap. 5.1.6 in [CP].
- Es gibt weitere solche Modi wie man z.B. im Buch von Bruce Schneier „ Applied Cryptography" nachlesen kann.
- Mit dem GMAC (Galois Counter Message Authentication Code) hat man einen Mode entwickelt, so dass auch die MAC Berechnung parallelisiert werden kann. Dieser Mode wird z.B. in IPSec verwendet.
- Weitere sind in [CP] in Kap. 12.5 erwähnt.
- Spannend sind kombinierte Modi (Verschlüsseln und MAC'en) als Ersatz des Galois Counter Mode (GCM), da dieser Schwächen enthält. Die Sieger des CAESAR Wettbewerbs (2023) heissen ACRON & AEGIS.
- **CAESAR = Competition for Authenticated Encryption: Security, Applicability and Robustness**
- Das ist aber alles für uns im Rahmen dieses Moduls nicht von Bedeutung.

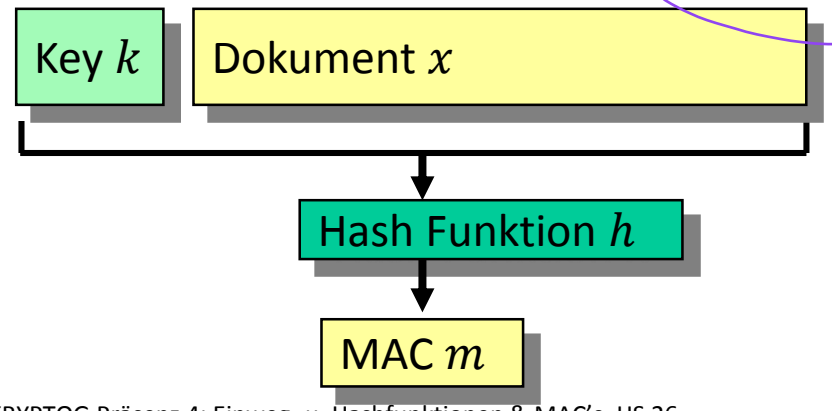
MAC mit „Keyed Hash Funct.“ = Geschichte HMAC

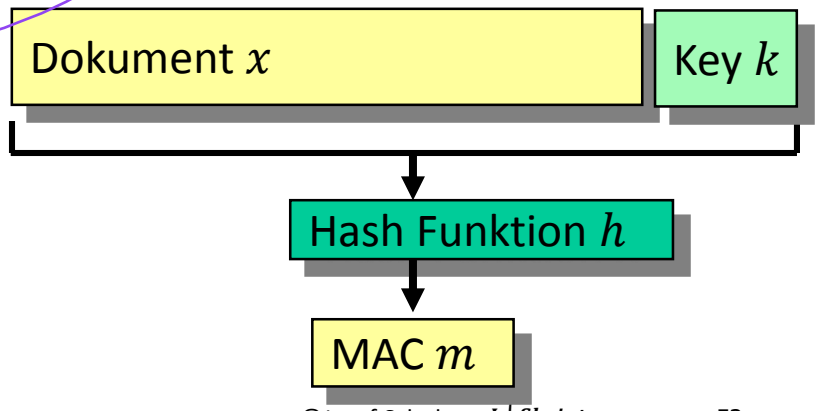
Schon früh kam der Wunsch auf, (grosse) Files gegen Abändern zu schützen. Oft enthielten diese Files keine Daten, die gegen Abhören geschützt werden mussten. D.h. eine Verschlüsselung – und damit z.B. Blockchiffren – waren nicht nötig. Dazu kommt, dass dieser Integritätsschutz nicht rechenintensiv sein sollte. Da kam die Idee schlüssellose Hashfunktionen zu nehmen und der Meldung M einen Schlüssel vor-, resp. anzuhängen und das Ganze zu hashen.

Die grundsätzliche Frage lautet:
Wie kann aus einer kryptographisch sicheren, schlüssellosen Hashfunktion eine Hashfunktion mit Schlüssel konstruieren?
Oder anders gesagt: Wie kann man aus einem MDC ein MAC machen?

MAC-Konstruktionen mit Hash-Funktionen, cf. Kap. 12.2 in [CP-D], resp. [RO2], Kap. 10.3.2. Dazu gibt es zwei naheliegende Ansätze:

- Der Secret-Prefix-MAC: $m = MAC_k(x) = h(k \parallel x)$ (linke Skizze)
- Der Secret-Suffix-MAC: $m = MAC_k(x) = h(x \parallel k)$ (rechte Skizze)

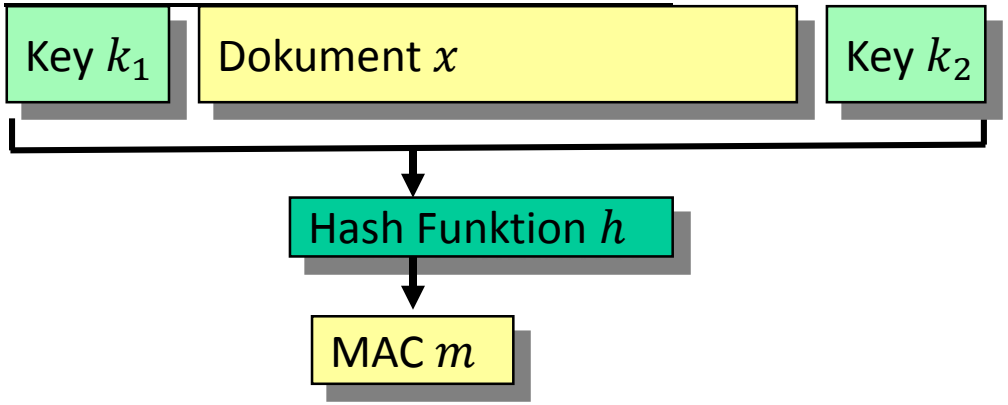




Eine dritte, kombinierte Variante

Aber schon bald wurden Angriffe bekannt, und die Variante mit zwei Schlüsseln k_1 und k_2 wurde definiert:

- Der Envelope-MAC: $m = MAC_{k_1,k_2}(x) =$

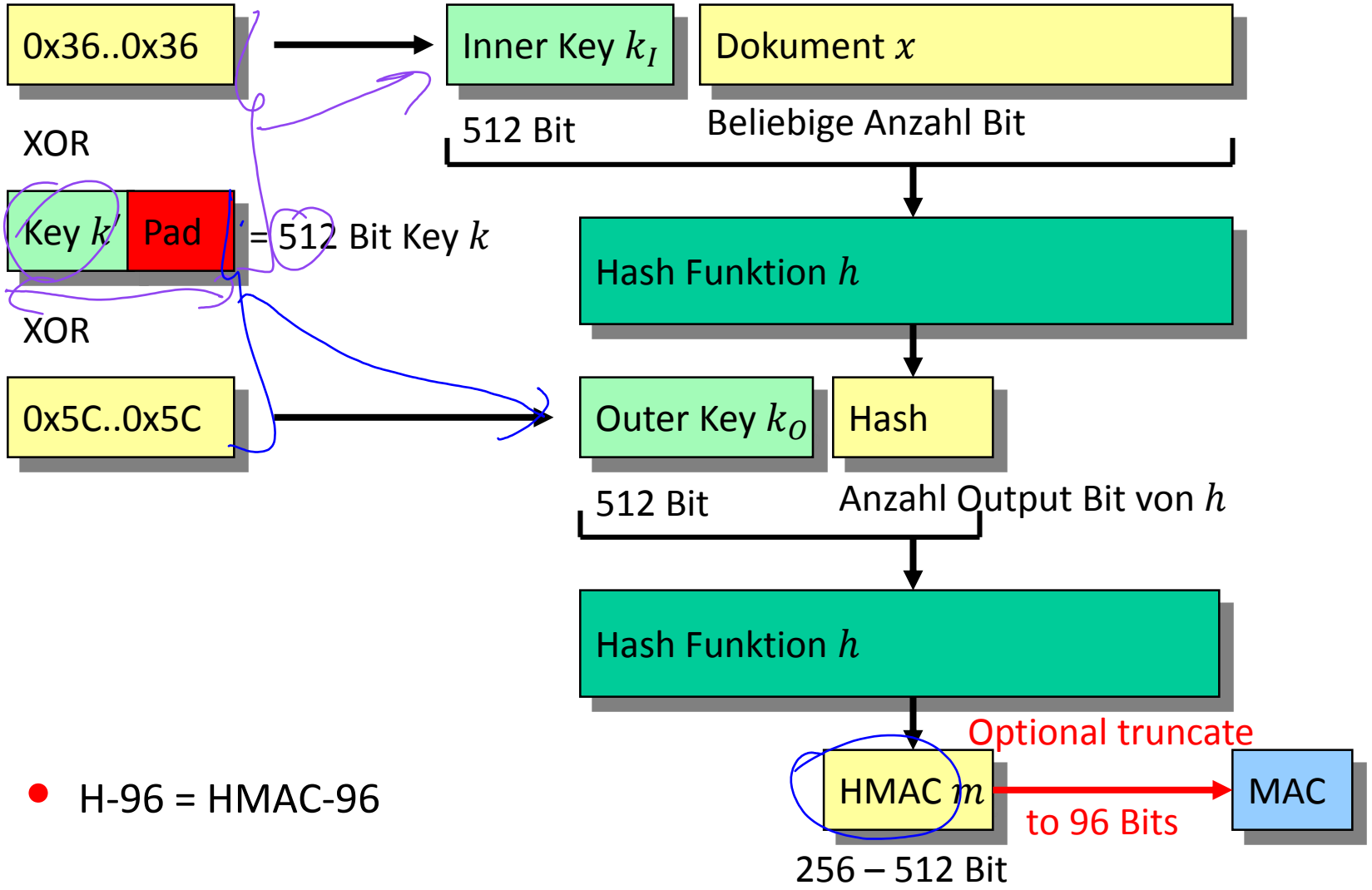


- Alle drei haben Schwächen, cf. [CP-D], Kap. 12.2.1 & 12.2.2, resp. [RO2], Kap. 10.3.2.
- Konsequenz: HMAC nach RFC 2104

Aufgabe 11

Schreiben Sie die Formel für $m = MAC_{k_1,k_2}(x) =$ _____ auf.

HMAC (HashMAC) nach RFC 2104



- H-96 = HMAC-96

HMAC (HashMAC) nach RFC 2104

Vorberechnung = Berechnung der 512 Bit grossen Schlüssel:

- ➔ 512 Bit der Bitfolge I: $I = ipad =$ „inner pad“ = 64mal die Bitfolge 00110110 = Hex 36
- ➔ 512 Bit der Bitfolge O: $O = opad =$ „outer pad“ = 64mal die Bitfolge 01011100 = Hex 5C
- ➔ Der Schlüssel k' wird durch Anhängen von Nullen aufgeblasen zum 512 Bit grossen k.
- ➔ Zum aufgeblasenen Schlüssel k werden die 2 Teile k_0 und k_i berechnet:
  - $k_O = k \oplus O = k \oplus opad$
  - $k_I = k \oplus I = k \oplus ipad$

Beispiel: Der Schlüssel $\boldsymbol{k'}$ sei 192 Bit gross.

- ➔ 512 Bit der Bitfolge I: $I = ipad =$ „inner pad“ = 64mal die Bitfolge 00110110 = Hex 36
- ➔ 512 Bit der Bitfolge O: $O = opad =$ „outer pad“ = 64mal die Bitfolge 01011100 = Hex 5C
- ➔ Der Schlüssel k' wird durch Anhängen von 512 – 192 = 320 Nullen aufgeblasen zum 512 Bit grossen k.
- ➔ Zum aufgeblasenen Schlüssel k werden die 2 Teile k_0 und k_i berechnet:
  - $k_O = k \oplus O = k \oplus opad$
  - $k_I = k \oplus I = k \oplus ipad$

HMAC (HashMAC) nach RFC 2104

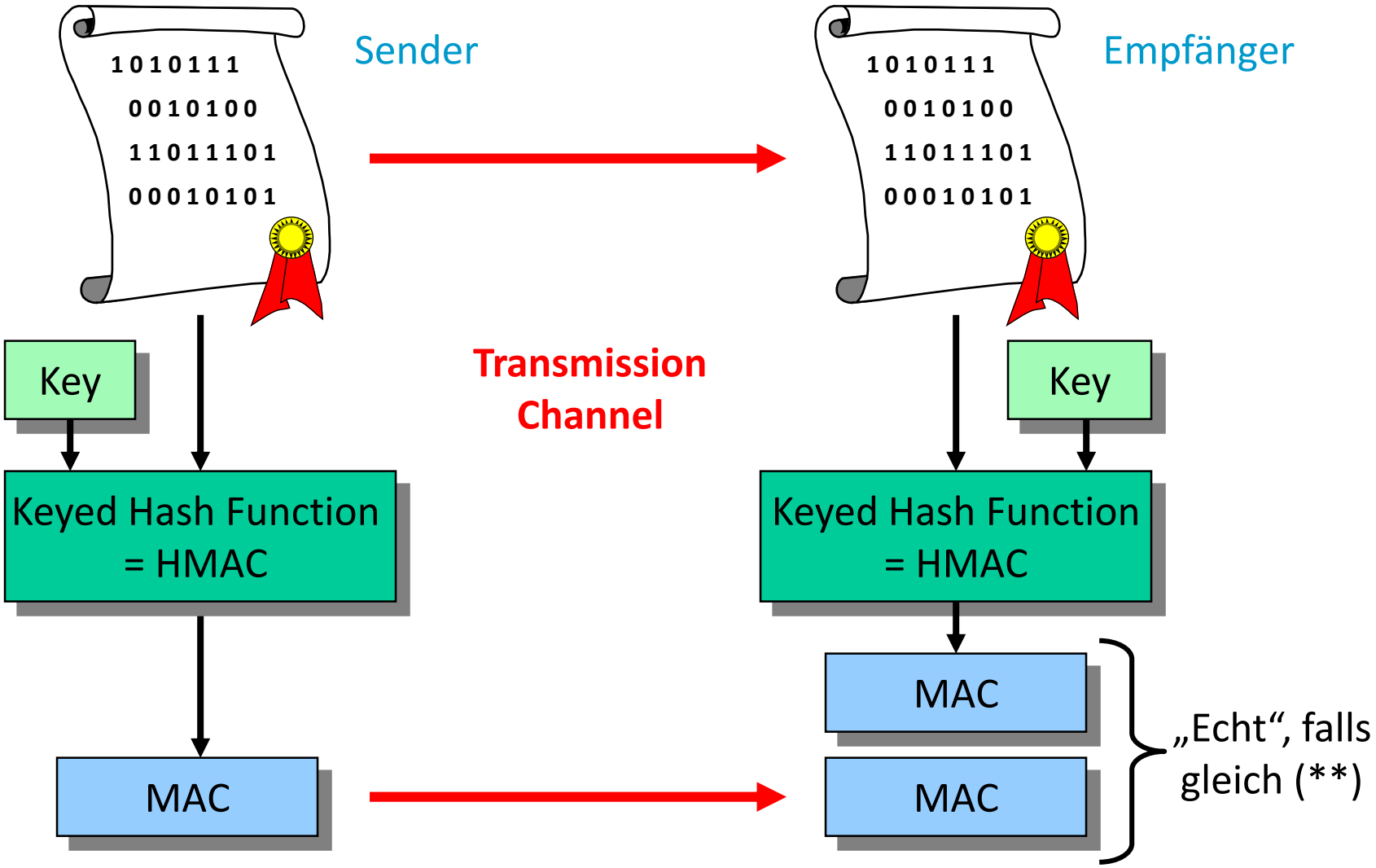
- Grundsätzlich kann der Key k' bis 512 Bit gross sein.
- Warum mit den Konstanten 36 und 5C gepadded wird, ist im RFC nicht ersichtlich.
- Gemäss BSI sind als Hashfunktionen nur noch SHA2 & 3 erlaubt.
- Wird oft in den Internetprotokollen wie TLS, IPSEC usw. verwendet, wobei in den neueren Versionen nun auch Kombi Modi wie GCM verwendet werden, cf. JS Skript, Kap. 8.6.
- Das Abschneiden auf 96 Bit ist eigentlich eine Schwächung, cf. JS Skript, Kap. 8.2.4.
- Das Abschneiden auf 96 Bit ist aber nicht vorgeschrieben.

- Der HMAC ohne Abschneiden:
  $m = HMAC_k(x) =$ _____

bis Hash2D

Aufgabe 12
Schreiben Sie die Formel für $m = HMAC_k(x) =$ _____ auf.

Verifikation mit HMAC



(*) zu 99,99… 9% echt falls gleich, aber zu 100% gefälscht, falls nicht gleich!

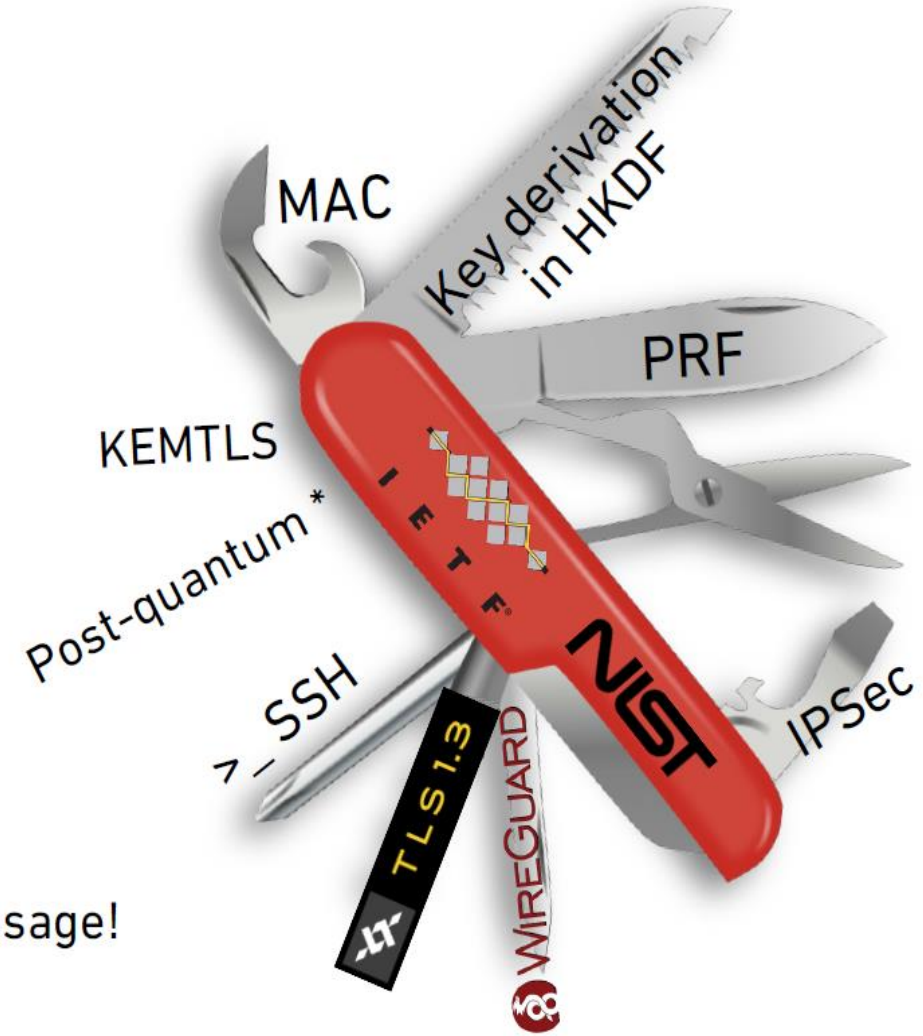
HMAC: the Swiss Army Knife of Crypto

HMAC [CRYPTO'96:BCK] is

- a hash-based MAC,
  - standardized,
  - provably secure,
  - versatile,
  - and widely used.

...as a PRF
[C'96:BCK, C'06:Bel, C'14:GPR].

This doesn't match current usage!



Matilda Backendal, ETHZ & Miro Haller, Uni San Diego, Vortrag im NIST Crypto Reading Club, April 3, 2024
https://mirohaller.com/talks/2024/04/thriving-between-theory-and-practice

Mathematische Beschreibung von MAC's

Analog zur Blockchiffre kann eine MAC Berechnung in einer mathematischen Form geschrieben werden. Es gibt einen IV der Länge i Bit (diese Länge kann auch 0 sein), eine beliebig grosse Meldung M der Länge * Bit („*“ bedeutet beliebige Länge) und Schlüssel K der Länge k Bit und Output MAC der Länge n Bit:

$$F: \{0,1\}^i \times \{0,1\}^* \times \{0,1\}^k \rightarrow \{0,1\}^n$$

$$IV \times M \times K \rightarrow MAC = F(IV, M, K)$$

MAC (Typ 2) und MDC (Typ 3) im Vergleich

WAS	Charakteristika
MAC = Typ 2	1. Muss (wegen dem Schlüssel) in einem Sicherheitsmodul implementiert sein. 2. Bietet (wegen dem Geheimnis) echte Integrität. 3. Für CBC-MAC gilt speziell: oft eher geringe Rechenrate. (Grund: sehr oft ist die Schnittstelle zum Sicherheitsmodul sehr langsam!!) 4. Es ist schwierig (resp. es muss schwierig sein), zwei Nachrichten mit dem gleichen MAC zu finden. 5. Ist ein Nachrichtenintegritäts- & „Modifikationserkennungswert“ (*).
Hashfunktion = MDC = Typ 3	1. Braucht kein Sicherheitsmodul. 2. Bietet alleine keine Integrität (in Kombination mit anderen Mechanismen kann aber Integrität erreicht werden, cf. Kap. 14.7.). 3. Ist so konzipiert, dass der Alg. auch in SW sehr, sehr schnell ist. 4. Es ist schwierig (resp. es muss schwierig sein), zwei Nachrichten mit dem gleichen MAC zu finden. 5. Hat beim Signieren die Rolle eines „Meldungsrepräsentanten“. 6. Ist nur ein „Modifikationserkennungswert“ im Sinne von unbeabsichtigtem Verändern („unintelligente“ Angreifer). Ein „intelligenter“ kann die Nachricht verändern und den neuen Hash berechnen.

(*) Der Begriff „Modifikationserkennungswert“ ist kein offizieller Begriff.

Reihenfolge von Verschlüsseln und MAC'en

Bez. der Reihenfolge von Verschlüsseln und MAC'en, resp. Signieren ist schon länger eine relativ hitzige Debatte am Laufen, cf. JS Skript Kap. 8.3.5.
Wenn auf dem Application Layer die Kryptographie eingesetzt wird, lautet die klassische Variante ist:

<u>Über den Klartext den MAC oder die Signatur berechnen und dann Verschlüsseln.</u>

Das sieht dann in einer Übersichtszeichnung wie folgt aus:

Header	**Body oder Meldungsinhalt**	**Trailer**
Mit MAC geschützt		**MAC ist im Trailer**
unverschlüsselt	**verschlüsselt**	**unverschlüsselt**
Enthält z.B. Routingdaten, Datum, Zeit & IV	Enthält die gesamten Nutzdaten	Enthält z.B. MAC und/oder verschlüsselte Schlüssel.
Unverschlüsselt und mit MAC geschützt.	Verschlüsselt und mit MAC geschützt.	In der Regel weder noch. (**)

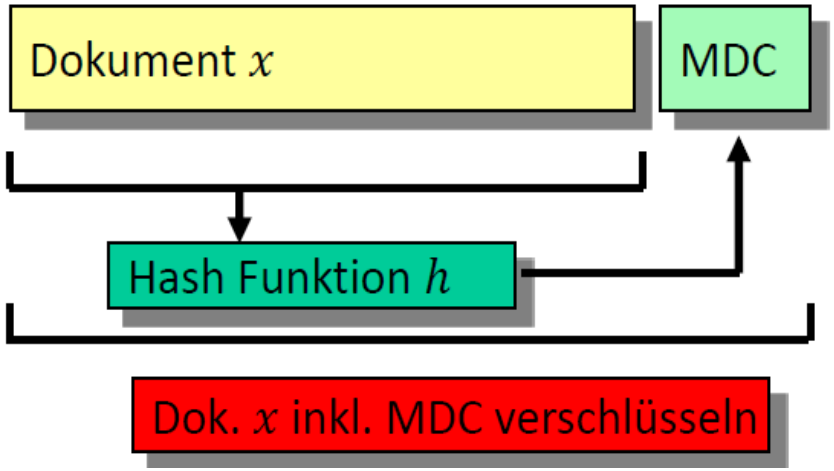
(**) Es gibt aber Protokolle, z.B. der Encapsulating Security Payload (ESP) bei IPSec, wo ein Teil des Trailers verschlüsselt und mit einem MAC versehen ist. In SSL 2.0 wird der MAC ebenfalls verschlüsselt der Grund liegt hier darin, dass in SSL 2.0 eine schwache MAC-Konstruktion gewählt wurde.

Kurzer Abriss zu kombinierten Verfahren

Ziel: Abhörsicherheit (Verschlüsselung) und Integritätsschutz in einem Schritt.

Erste Ansätze: An Meldung einen Hash (MDC) anhängen und verschlüsseln.

- Zuerst wird ein MDC über die Meldung gerechnet.
- Dieser wird an die Meldung angehängt.
- Die ganze Meldung (inkl. MDC) wird nun verschlüsselt und verschickt.



Bemerkungen:

1) Verschlüsseln und MDC Berechnung bieten je alleine keine Integrität, die Kombination aber schon.
2) Eine Variante war, nur die Meldung zu verschlüsseln und den MDC anzuhängen.
3) Es zeigte sich bald, dass beide kombinierten Verfahren Schwächen haben, 2) kam noch schlechter weg.

Heute: Kombi-Mode u.a. GMAC, CMAC und die neu (2023) definierten ACRON & AEGIS.

Authenticated Encryption, JS Skript Kap. 8.3.8 (#)

Seit der zivilen Nutzung der Kryptographie und der „Geburt“ der Integrität von G. Simmons 1970 wurden die Mechanismen der Integrität (CBC-MAC) und Verschlüsselung kombiniert. Ab ca. dem Jahre 2000 entwickelte sich daraus folgenden Themenbereiche:

- Attacken auf MAC Algorithmen und Entwerfen von entsprechenden Gegenmassnahmen.
- Diskussion um die Reihenfolge von MAC‘en und Verschlüsseln (Vor- & Nachteile sowie Attacken und Gegenmassnahmen).
- MAC’en und Verschlüsselen in einem Schritt (Kombi Modi).
- Neue Begrifflichkeiten, wobei – wie leider auch in anderen Fällen – die Definitionen nicht einheitlich sind.

Vieles davon wurde (oder wird noch) in diesem Modul angesprochen. Das Kap. 8.3.8 Authenticated Encryption soll dazu dienen bei Bedarf sich entsprechend in diese Thematik einarbeiten zu können.
Der Inhalt von Kap. 8.3.8 ist aber ***nicht*** prüfungsrelevant!!

Bemerkung:
In den Beschreibungen der Projektideen (SecProj, WiPro, BAA), Kap. 25 ist dazu eine Arbeit ausgeschrieben.

CBC-MAC (HMAC)
& digitale Signaturen
in Authentication Schemes
= Ausblick auf KRYPTOE

User Authentication → siehe unsere 8 Angriffe

Angriff	Alice (*)	Eve	Bob
Abhören der Meldung (Confidentiality)		X	
Verändern der Meldung (Integrity)		X	
Eine erfundene Meldung einspielen (Insertion)		X	
Eine Meldung abfangen u. später wieder einspielen, resp. mehrmals zu schicken (Replay)		X	
Löschen von Meldungen (Delete)		X	
Abstreiten die Meldung geschickt zu haben (Non repudiation of origin)	X		
Abstreiten die Meldung erhalten zu haben (Non repudiation of receipt)			X
Sich für jemanden anders ausgeben (Masquerade)		X	

Aufgabe 13

a) Welche(r) Angriff(e) muss/müssen in Kombination mit einem kryptographischen Mechanismus und einem Protokoll verhindert werden?

b) Welche(r) Angriff(e) benötigt eine User Authentication, damit er/sie verhindert wird/werden?

User Authentication → KRYPTOE „Protokolle“



“On the Internet, nobody knows you’re a dog.”

- Username / Password
  Dictionary Attacks
- One-Time Passwords
  Token: SecureID, etc.
- Symmetric Algorithms
  C-R-Protocols
- Public Key Algorithms
  Smartcards, Certificates, Public Key Infrastructure, C-R-Protocols
- Biometrical Methods
  Fingerprint, Iris-Scan, Voice, Face, Hand, etc.

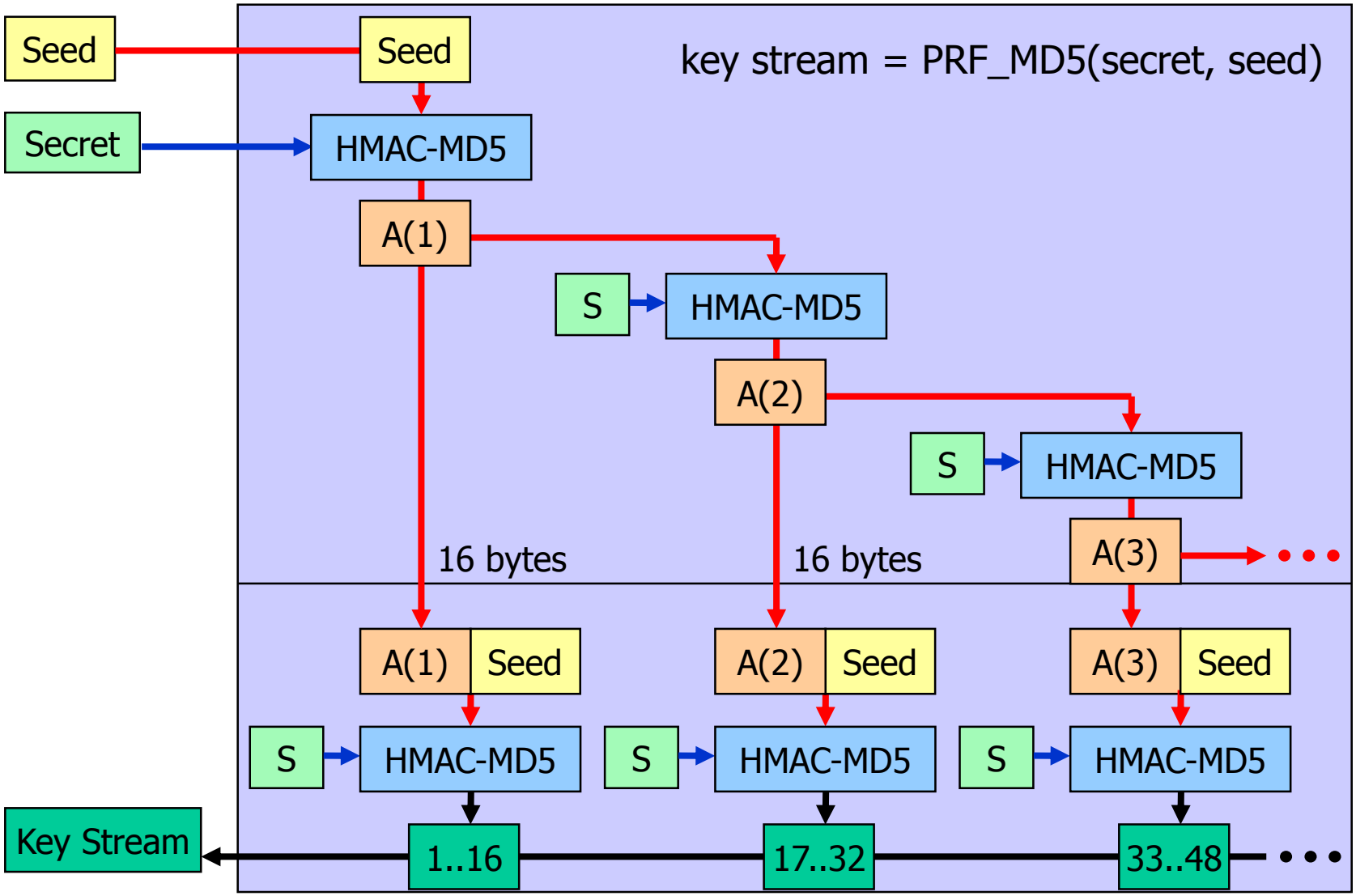
MDC

in der Schlüsselgenerierung (#)

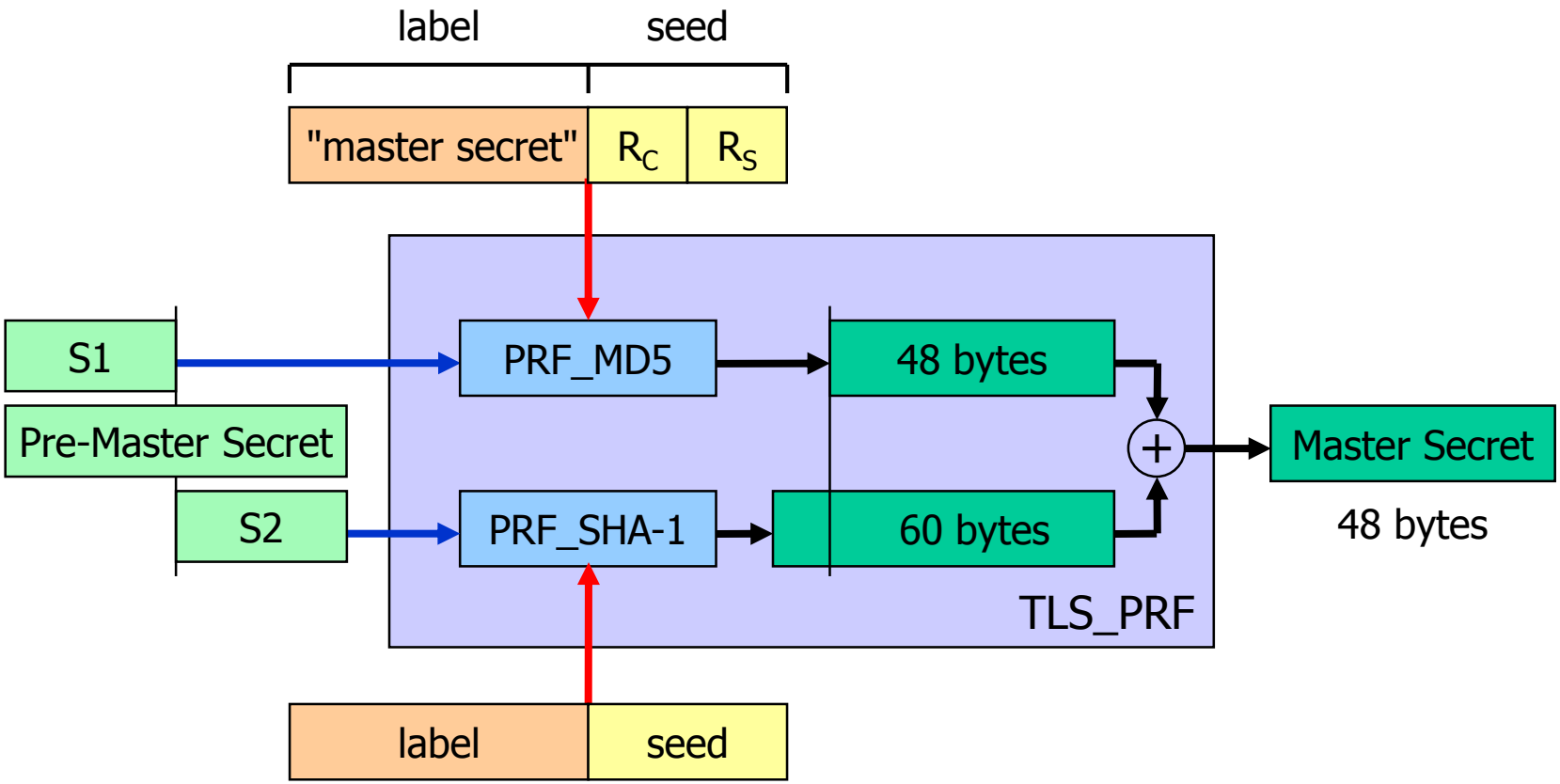
Hashfunktionen in der Schlüsselgenerierung (#)

- In der elektronischen Zahlungswelt wird die Schlüsselgenerierung oft mit Blockchiffren gemacht, cf. Präsenz 2, Kap. 8.23 & 8.4. Wir gehen aber nicht weiter darauf ein.
- In der Netzwerksicherheit (z.B. TLS) wird die Schlüsselgenerierung in vielen Fällen mit Hashfunktionen gemacht.

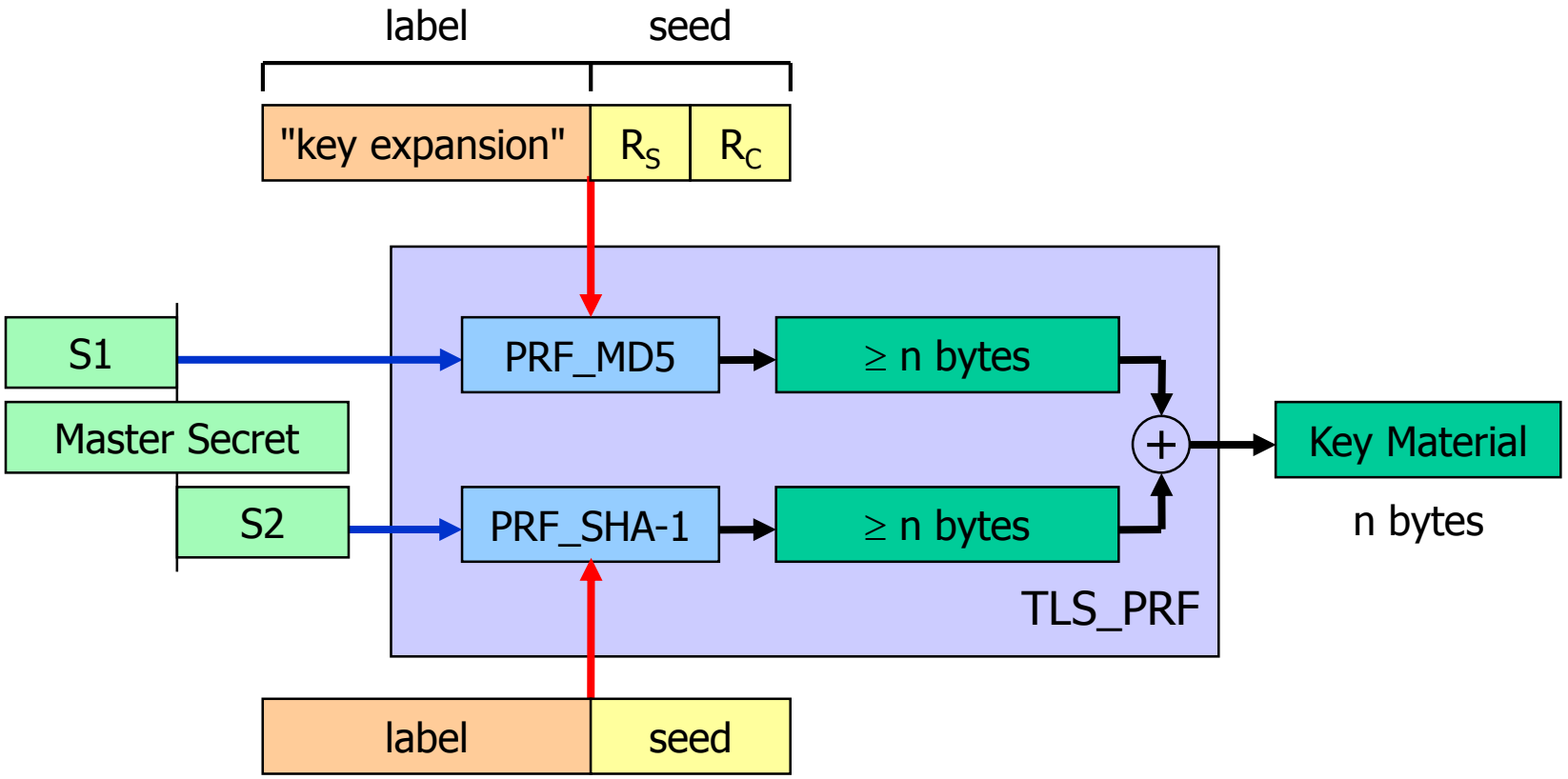
Pseudo Random Function (PRF) (#)



Computing the Master Secret (#)



Generating Key Material (#)



key stream = TLS_PRF(secret, label, seed)

Warum besteht ein Unterschied zw. Unterschrift und HMAC Generierung und Schlüsselerzeugung?

- Frage: Warum besteht ein Unterschied zw. Unterschrift und HMAC Generierung und Schlüsselerzeugung?
- Antwort:
  - Eine Unterschrift muss in 15 – 20 Jahren noch gültig, resp. integer sein.
  - Ein Integritätsschutz einer Meldung ist sehr viel kurzlebiger, in der Regel nach Erhalt der Meldung nicht mehr relevant.
  - Dito für einen Sessionschlüssel (man beachte auch, dass z.T. während einer Verbindung ein Schlüsselwechsel passiert).

Kap. 8.4

RANDOMFUNKTIONEN

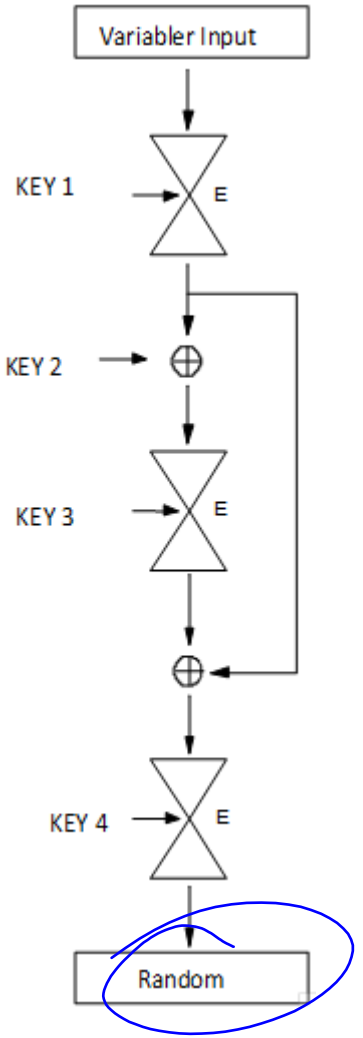
MIT …

- **… BLOCKCHIFFREN**
- **… HASHFUNKTIONEN**

Beispiel einer Randomfkt. mit Blockchiffren

Bemerkungen:

- Die Version mit nur 2 verschiedenen Schlüsseln (Key 3 = Key 4 = Key 1) war früher im Annex von ISO 8732 standardisiert. Der Standard – und damit der Annex – wurde zurückgezogen.
- ***Nie*** Hardware Zufallszahlengeneratoren (z.B. Rauschdioden) alleine verwenden.
- Wenn ein Hardware Zufallszahlengenerator (z.B. Rauschdiode) zur Verfügung steht, dann soll er nur benutzt werden, um den obigen Input zu verbessern.
- **Vorschlag:** Input = (Datum || Zeit) $\oplus$ Output der „Rauschdiode“
- **Analyse des Vorschlags:** Sollte die Rauschdiode kleine Zyklen liefern, ist der Input immer noch so gut, wie (Datum || Zeit) alleine.
- Wird eine solche Funktion zur Erzeugung von Schlüsseln gebraucht, dann nennt man sie auch ***KDF (Key Derivation Function).***
- Im Rahmen des Key Managements (cf. KRYPTOE) werden wir auf diese Funktion zurückkommen.



Beispiel Randomfkt. Math. dargestellt

Eine so kompliziertere Zusammensetzung wird am besten in Teilschritte aufgeteilt:

Schritt 1: $\boldsymbol{A = E(In, K_1)}$

Schritt 2: $\boldsymbol{B = A \oplus K_2 = E(In, K_1) \oplus K_2}$

Schritt 3: $\boldsymbol{C = E(B, K_3) = E(A \oplus K_2, K_3) = E\big((E(In, K_1) \oplus K_2), K_3\big)}$

Schritt 4: $\boldsymbol{D = C \oplus A = E(A \oplus K_2, K_3) \oplus A = E\big((E(In, K_1) \oplus K_2), K_3\big) \oplus E(In, K_1)}$

Schritt 5: $\boldsymbol{Out = E(D, K_4) = E(C \oplus A, K_4) = E(E(A \oplus K_2, K_3) \oplus A, K_4) = E\big(E\big((E(In, K_1) \oplus K_2), K_3\big) \oplus E(In, K_1), K_4\big)}$

Eine so kompliziertere Zusammensetzung wird am besten in Teilschritte aufgeteilt:

Schritt 1: $A = E(In, K_1)$

Schritt 2: $B = A \oplus K_2 = E(In, K_1) \oplus K_2$

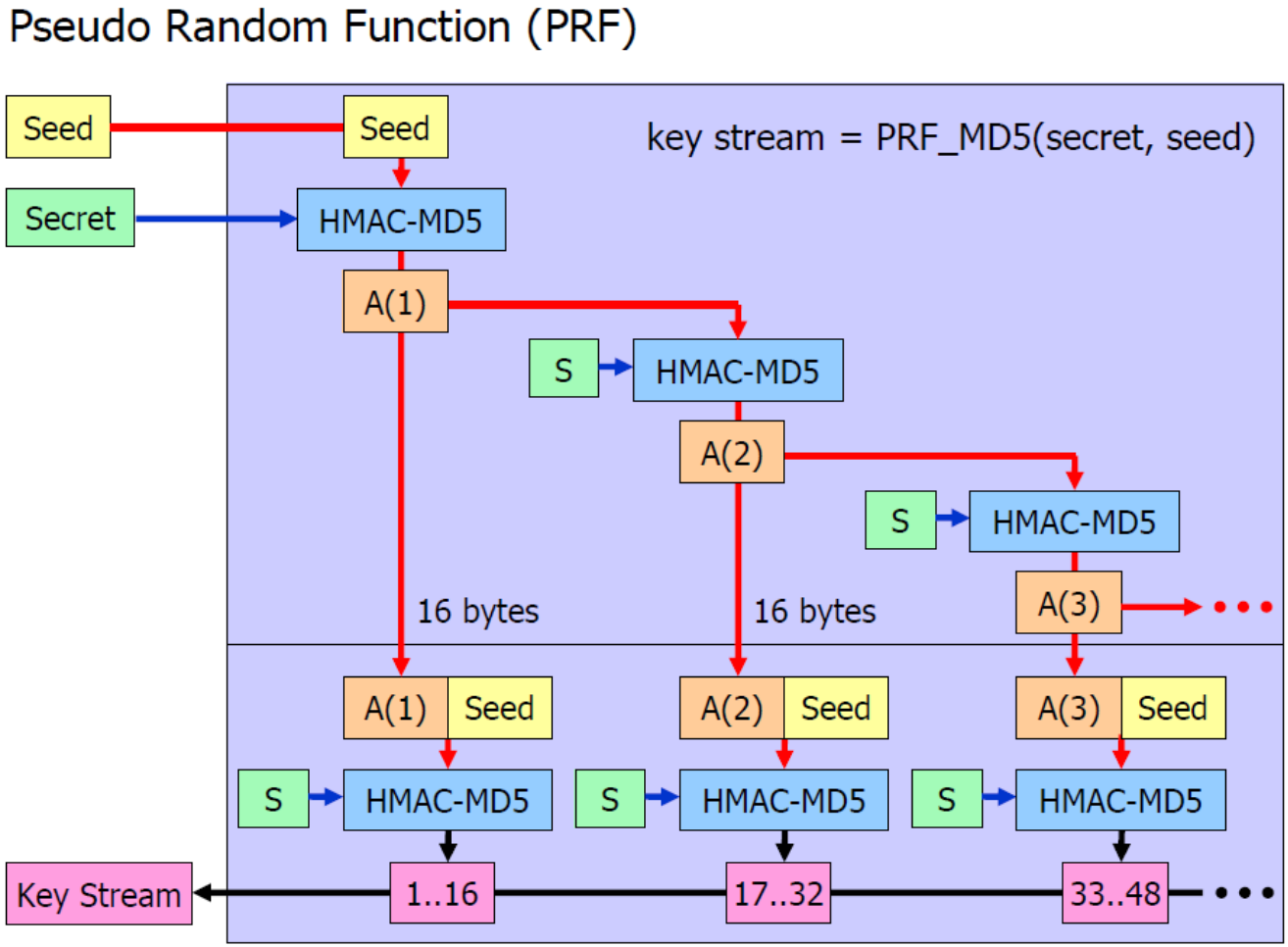
Schritt 3: $C = E(B, K_3) = E(A \oplus K_2, K_3) = E\big((E(In, K_1) \oplus K_2), K_3\big)$

Schritt 4: $D = C \oplus A = E(A \oplus K_2, K_3) \oplus A = E\big((E(In, K_1) \oplus K_2), K_3\big) \oplus E(In, K_1)$

Schritt 5: $Out = E(D, K_4) = E(C \oplus A, K_4) = E(E(A \oplus K_2, K_3) \oplus A, K_4) = E\big(E\big((E(In, K_1) \oplus K_2), K_3\big) \oplus E(In, K_1), K_4\big)$

Beispiel einer Randomfkt. mit Hashfkt. (#)

(#) Nicht Prüfungsstoff, nur Wissen, dass es solche Randomfunktionen gibt.
Das gilt auch für die Folien 69 – 71. Die untenstehende Folie ist eine Kopie der Folie 69.

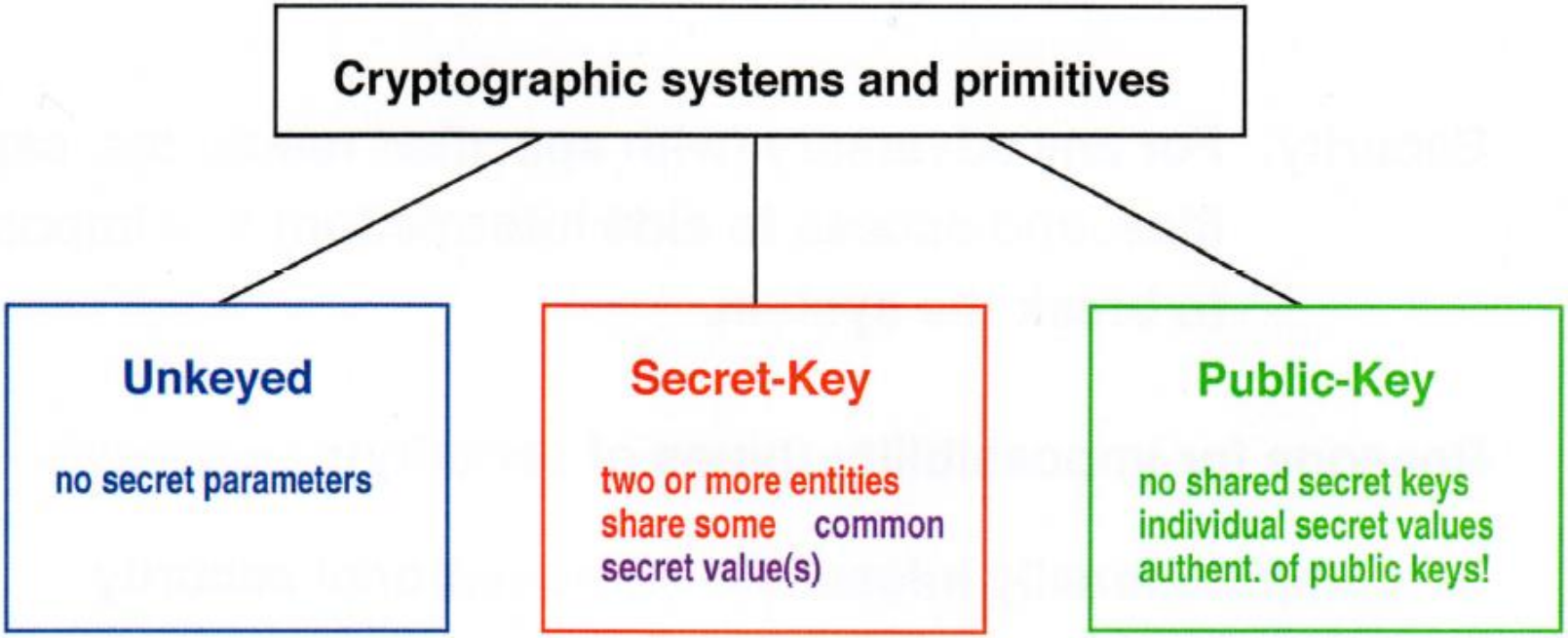


Kap. 27
EINWEGFUNKTIONEN
ÜBERSICHT & VORBEREITUNG AUF ASYMMETRISCHE VERFAHREN

Wird ev. nur kurz besprochen, ev. auch weggelassen und erst zu Beginn der Präsenz 7 präsentiert!

Einleitung

Neben der ***Secret und Public Key*** Kryptographie betrachtet Prof. Dr. U. Maurer in seiner erweiterten Übersicht die ***Unkeyed*** (also schlüssellose) Kryptographie. Wobei wir im Verlaufe des Moduls gesehen haben, dass es sehr wohl auch ***keyed*** Einwegfunktionen gibt.



Schreibweise

Die (verschiedenen) Schreibweisen vom Begriff „Einwegfunktion“ in deutschsprachigen Büchern:

- **Einwegfunktion (wir benutzen diese Schreibweise, sie kommt am häufigsten vor).**
- Einweg-Funktion
- Ein-Weg-Funktion
- Es ist nicht auszuschliessen, dass es weitere gibt.

In der englischen Literatur wird mehrheitlich der Begriff „one-way function“ verwendet.

Bemerkungen:

1) Es ist mir kein (deutsches oder englisches) Buch bekannt, dass das Thema „Einwegfunktion“ oder „one-way function“ zentral in einem Kapitel zusammengefasst thematisiert.
2) Wir haben uns nun verschiedentlich und in unterschiedlichen Themen (z.B. asym. Krypto, Randomfunktionen, Schlüsselableitungsfunktionen mit Blockchiffren (Kap. 8.4.1, resp. Kap. 9) und Hashfunktionen (Kap. 14) mit Einwegfunktionen beschäftigt.

Definition

Ich zitiere hier die Definition aus [CP-D], S. 177. CP spricht hier auch von einer informellen Definition.

Definition 6.1 (Einwegfunktion)
Eine Funktion $f(\cdot)$ ist eine Einwegfunktion, wenn:

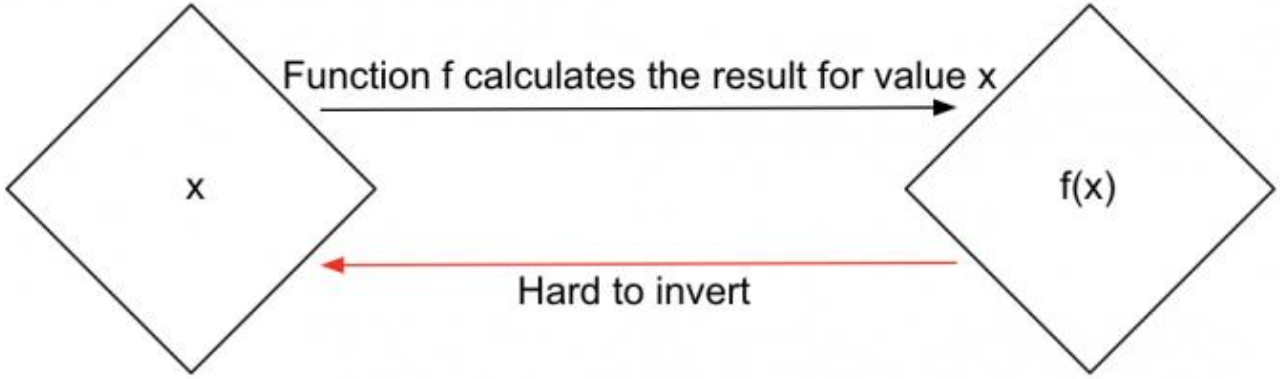
1. $y = f(x)$ rechentechnisch einfach und
2. $x = f^{-1}(y)$ technisch unmöglich zu berechnen ist.

- Offensichtlich sind die Adjektive ***einfach*** und ***unmöglich*** nicht sehr exakt.
- Mathematische Sicht: ***einfach*** = ***Berechnung in polynomialer Zeit möglich***.
- Also:
  - Berechnung von $y = f(x)$ so schnell sein, dass keine unverhältnismäßig langen Verzögerungen entstehen.
  - Berechnung der Umkehrfunktion $x = f^{-1}(y)$ muss derart aufwendig sein, dass selbst mit den besten Algorithmen und sehr vielen Rechenressourcen nicht in einer akzeptablen Zeit durchgeführt werden kann, z.B. 100.000 Jahre oder länger dauern würde (*).

(*) Anmerkung JS:
Dies erreicht man (als notwendige Bedingung), dass sich die Rechenzeit der Umkehrfunktion pro Bit exponentiell erhöht.

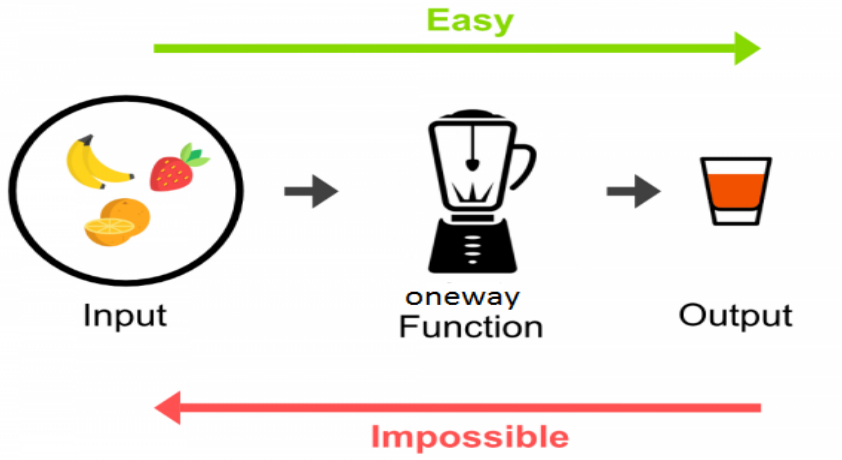
Bilder von Einwegfunktionen

Die typische mathematische Zeichnung sieht in etwa wie folgt aus:



Es gibt aber auch sehr anschauliche:





Typen von Einwegfunktionen

- Es gibt zunächst Einwegfunktionen …
  - ***… mit*** Trapdoor (d.h. mit einem Trick kann $x = f^{-1}(y)$ ebenfalls schnell berechnet werden).
  - ***… ohne*** Trapdoor (d.h. niemand kennt einem Trick, um $x = f^{-1}(y)$ schnell berechnen zu können).

- Einwegfunktionen in der asymmetrischen Kryptographie, z.B.
  - Das Faktorisierungsproblem: für eine grosse Zahl N die Primfaktoren p und q zu finden, so dass $N = p \cdot q$
  - Das Berechnen von der diskreten e-ten Wurzel mod N: $x = \sqrt[e]{y}\ mod\ N$
  - Das Berechnen des diskreten Logarithmus mod p:

$$y = g^x\ mod\ p \Rightarrow x = \log_g y\ mod\ p$$

- Kryptographisch sichere Hashfunktionen …
  - … ohne Schlüssel wie SHA-3 usw.
  - … mit Schlüssel wie (CBC-)MAC oder HMAC

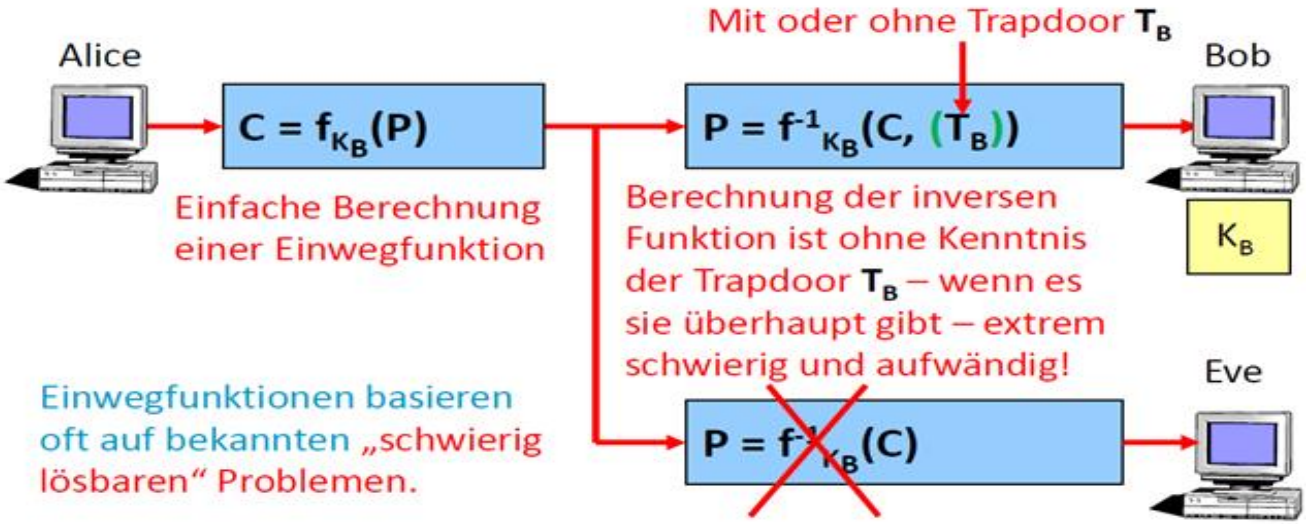
- Schlüsselableitungsfunktionen (KDF = Key Derivation Function) oder Randomfunktionen mit Blockchiffren, wie z.B. in Kap. 8.4.

Aufgabe 14 Warum ist eine Prüfziffer eine Hashfunktion, aber nicht eine Einwegfunktion ist.

Einwegfunktionen in der asym. Kryptographie

- Die Erfinder
  - Whitfield Diffie und Martin Hellman 1976
  - Ralph Merkle 1978





- Einwegfunktionen mit Trapdoor, d.h. es gibt einen „Trick“, so dass die „unmögliche“ Berechnung von $x = f^{-1}(y)$ gleich einfach ist wie $y = f(x)$. Die Sicherheit liegt hier ja genau darin, dass dieser „Trick“ nur einer Stelle bekannt ist.
- Einwegfunktionen ohne Trapdoor, d.h. es gibt keinen Trick. D.h. die Berechnung von $x = f^{-1}(y)$ ist für alle schwierig! Die Sicherheit liegt hier ja genau darin, dass es ***keinen*** „Trick“ gibt.
- Eine solche Einwegfunktion kann eine Einwegpermutation sein.

Einwegfunktionen mit/ohne Trapdoor

Einwegfunktion mit Trapdoor

Beim RSA: Die e-te Wurzel mod N berechnen: ***mit*** Trapdoor

- Das Potenzieren $y = f(x) \equiv x^e \; mod \; N$ ist rechenintensiv, aber einfach.
- Das Inverse $x \equiv f^{-1}(x) \equiv y^{1/e} \equiv \sqrt[e]{y} \; mod \; N$ ist extrem schwierig.
- Dabei ist das Berechnen der „e-ten Wurzel mod N“ das eine Problem, das Faktorisieren von $N = p \cdot q$ das andere. Der RSA kann gebrochen werden, wenn ***nur*** eines der zwei Probleme gelöst ist.

Einwegfunktion ohne Trapdoor

Beim Diffie-Hellman: Der Logarithmus mod p berechnen: ***ohne*** TP

- Die Potenz $y = f(x) \equiv a^x \; mod \; p$, p prim ist rechenintensiv, aber einfach.
- Das Inverse (der diskrete Logarithmus) $x \equiv f^{-1}(x) \equiv \log_a y \; mod \; p$ ist ***für alle*** – also auch für Alice und Bob – extrem schwierig.

Bei Elliptischen Kurven: ein anderes disk. Log. Problem: ***ohne*** TP

- Die Multiplikation $Q = k \cdot P$, k eine unbekannte Zahl, P und Q bekannte Punkte einer Elliptischen Kurve.
- Das Bestimmen von k aus $Q = k \cdot P$ ist ebenfalls ***für alle*** extrem schwierig, dabei um handelt es sich um eine andere Art von diskretem Logarithmus.
- Die Punkt-Division von $\boldsymbol{k = \frac{Q}{P}}$ gibt es in diesem Sinne ***nicht***.

Bemerkung: Diese Einwegfunktionen sind genau genommen Einwegpermutationen, cf. Kap. 27.4.3, resp. Nachtrag.

Hashfunktionen als Einwegfunktionen

- Hashfunktionen ohne Schlüssel (MDC's) als Einwegfunktionen, cf. Kap. 27.5.1, sowie Kap. 14.2.
- Hashfunktionen mit Schlüssel (MAC, HMAC) als Einwegfunktionen, cf. Kap. 27.5.2, sowie Kap. 8.3 & 14.3 – 14.6.
- **Nachbearbeitung:** Bitte lesen Sie das Kap. 27.5 im JS Skript.

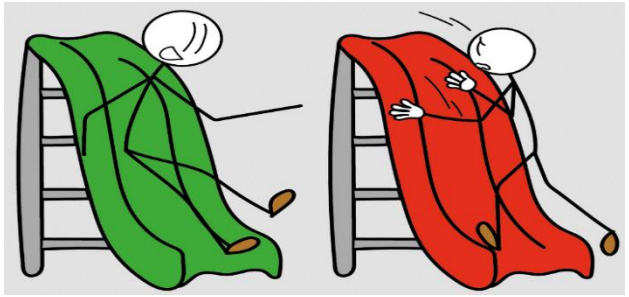
Blockchiffren als Einwegfunktionen

- Einführende Gedanken in Kap. 27.6.1.
- Blockchiffren als Schlüsselableitungsfunktioen (KDF = Key Derivation Function) in Kap. 27.6.2 & 8.4.1.
- **Nachbearbeitung:** Bitte lesen Sie das Kap. 27.5 im JS Skript.

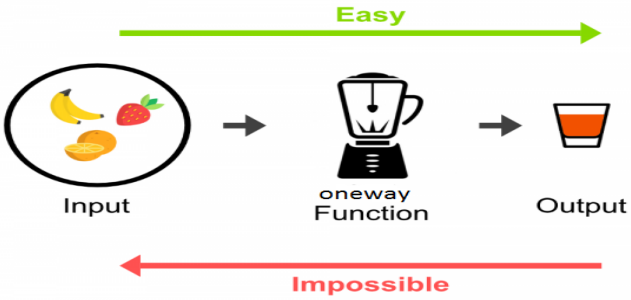
Aufgabe 15 zu Einwegfunktionen

Geben Sie an bei den folgenden Einwegfunktion an, ob sie eher „mit“ oder eher „ohne“ Trapdoor sind. Geben Sie im Falle „mit“ den Trick an.

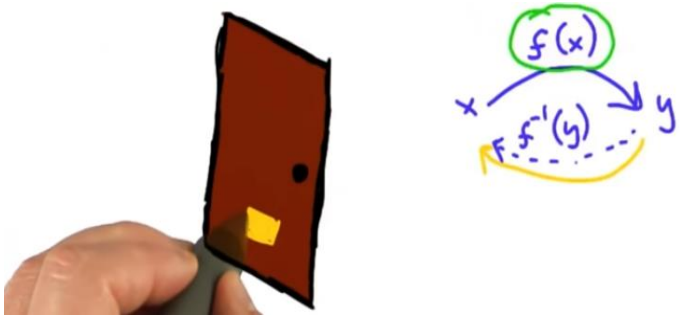
bis 17h45



Einwegfunktion _____ Trapdoor



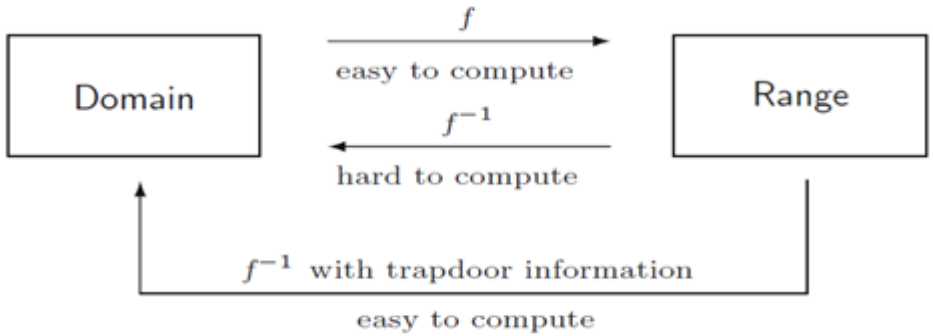
Einwegfunktion _____ Trapdoor



Einwegfunktion _____ Trapdoor

Aufgabe 16

Figure: Illustration of an one-way trapdoor function



NR	Aufgabe	Auswahl
a)	Welche Berechnungen sind im Rahmen von Einwegfunktionen von der Art „f easy to compute“?	☐ Die Ver- resp. Entschlüsselung mit XOR. ☐ Die Ver- resp. Entschlüsselung mit einer Blockchiffre. ☐ Die Berechnung der Werte, die beim DH Schlüsselaustausch über die Leitung geschickt werden. ☐ Die Berechnung des sogenannten diskreten Logarithmus beim Diffie-Hellman Schlüsselaustausches. ☐ Das Berechnen von $c = m^e \ mod \ N$ beim RSA. ☐ Das Berechnen von $m = \sqrt[e]{c} \ mod \ N$ beim RSA. ☐ Die Multiplikation $r \cdot P$ bei den Elliptischen Kurven. ☐ Das Bestimmen vom unbekannten k bei Elliptischen Kurven, falls $Q = k \cdot P$. ☐ Keine der Angaben ist zutreffend.
b)	Welche Berechnungen sind im Rahmen von Einwegfunktionen von der Art „f^{-1} hard to compute“?	☐ Die Ver- resp. Entschlüsselung mit XOR. ☐ Die Ver- resp. Entschlüsselung mit einer Blockchiffre. ☐ Die Berechnung der Werte, die beim DH Schlüsselaustausch über die Leitung geschickt werden. ☐ Die Berechnung des sogenannten diskreten Logarithmus beim Diffie-Hellman Schlüsselaustausches. ☐ Das Berechnen von $c = m^e \ mod \ N$ beim RSA. ☐ Das Berechnen von $m = \sqrt[e]{c} \ mod \ N$ beim RSA. ☐ Die Multiplikation $r \cdot P$ bei den Elliptischen Kurven. ☐ Das Bestimmen vom unbekannten k bei Elliptischen Kurven, falls $Q = k \cdot P$. ☐ Keine der Angaben ist zutreffend.
c)	Welche Berechnungen sind im Rahmen von Einwegfunktionen von der Art „f^{-1} with trapdoor information easy to compute“?	☐ Die Ver- resp. Entschlüsselung mit XOR. ☐ Die Ver- resp. Entschlüsselung mit einer Blockchiffre. ☐ Die Berechnung der Werte, die beim DH Schlüsselaustausch über die Leitung geschickt werden. ☐ Die Berechnung des sogenannten diskreten Logarithmus beim Diffie-Hellman Schlüsselaustausches. ☐ Das Berechnen der Potenzen beim RSA. ☐ Das Berechnen des ursprünglichen Wertes beim RSA. ☐ Die Multiplikation $r \cdot P$ bei den Elliptischen Kurven. ☐ Das Bestimmen vom unbekannten k bei Elliptischen Kurven, falls $Q = k \cdot P$. ☐ Keine der Angaben ist zutreffend.

Basis-Test Präsenz 4

Aussage	Richtig oder falsch?	Begründung
Eine kryptographisch sichere Hashfunktion ist auch eine Einwegfunktion.	☐ richtig ☐ falsch	
Eine Einwegfunktion ist auch eine kryptographisch sichere Hashfunktion.	☐ richtig ☐ falsch	
Eine Prüfziffer schützt z.B. vor versehentlichem Verschreiben.	☐ richtig ☐ falsch	
Für SHA-3 kann man vier zentrale Sicherheitseigenschaften aufzählen.	☐ richtig ☐ falsch	
Eine Hashfunktion (= MDC) muss seit 2023 mindestens 240 Bit Output haben.	☐ richtig ☐ falsch	
Den SHA3-224 darf noch lange für Signaturen verwendet werden.	☐ richtig ☐ falsch	
Die wichtigste Funktion eines Hashes ist die Komprimierung einer Meldung.	☐ richtig ☐ falsch	
Der Kollisionsangriff ist weniger aufwendig, aber auch weniger realistisch als ein Pre-Image Angriff.	☐ richtig ☐ falsch	
Der Pre-Image Angriff ist weniger aufwendig, aber auch weniger realistisch als ein Kollisionsangriff.	☐ richtig ☐ falsch	
Den CBC-MAC gibt es in verschiedenen Ausprägungen.	☐ richtig ☐ falsch	
Ein HMAC und ein CBC-MAC haben unterschiedliche Anwendungszwecke.	☐ richtig ☐ falsch	

Basis-Test Präsenz 4, Fortsetzung

Aussage	Richtig oder falsch?	Begründung
Mit Blockchiffren kann man Zufallszahlen erzeugen.	☐ richtig ☐ falsch	
Mit Hashfunktionen kann man keine Zufallszahlen erzeugen.	☐ richtig ☐ falsch	
Die asymmetrische Kryptographie basiert vornehmlich auf Hashfunktionen.	☐ richtig ☐ falsch	
Die asymmetrische Kryptographie basiert vornehmlich auf Einwegfunktionen.	☐ richtig ☐ falsch	
Einwegfunktion mit Trapdoor ist ein Widerspruch in sich.	☐ richtig ☐ falsch	
Das Ziehen einer e-ten Wurzel mod N ($N = p \cdot q$ mit p, q grosse Primzahlen) ist eine Einwegfunktion ohne Trapdoor.	☐ richtig ☐ falsch	
Der diskrete Logarithmus ist eine Einwegfunktion ohne Trapdoor.	☐ richtig ☐ falsch	
Sei g ein Generator in $\mathbb{Z}_p^*$ und p eine Primzahl, dann ist $y \equiv g^x \; mod \; p$ eine Einwegpermutation (*).	☐ richtig ☐ falsch	

(*) Diese Aufgabe kann erst nach dem Lesen des Nachtrags Kap. 27.4.3 «Einwegpermutationen» gelöst werden.

Nachtrag
Kap. 27.4.3
EINWEGPERMUTATIONEN

Wird ev. erst zu Beginn der Präsenz 7 präsentiert!

Was zum Kuckuck sind Einwegpermutationen?

Permutation

Die Farben blau, grün, rot kann man auf $3! = 6$ Möglichkeiten anordnen:

(B, G, R), (B, R, G), (G, R, B), (G, B, R), (R, B, G), (R, G, B)

Wir betrachten das RSA-System mit

$N = 3 \cdot 11$ und $e = 13 \Rightarrow d = 13^{-1}\ mod\ \varphi(N) = 17 = 13^{-1}\ mod\ 20$

Nun kann man alle Werte $x \in \mathbb{Z}_{33} = \{0, \dots, 32\}$ mit $y \equiv x^{13}\ mod\ 33$ verschlüsseln und y nimmt auch alle Werte in $\mathbb{Z}_{33}$ (sonst könnte man ja nicht entschlüsseln).

$x \in \mathbb{Z}_{33}$	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**
$\boldsymbol{x^{13}\ mod\ 33}$	0	1	8	27	31	26	18	13	17	3	10	11	12	19	5	9	4

$x \in \mathbb{Z}_{33}$	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**
$\boldsymbol{x^{13}\ mod\ 33}$	29	24	28	14	21	22	23	30	16	20	15	7	2	6	25	32

Frage: Wie viele solche Anordnungen gibt es?

Antwort: Theoretisch $33! \approx 8{,}7 \cdot 10^{36}$, aber die allermeisten sind nicht in der Form $\boldsymbol{y = x^e\ mod\ 33}$ für ein geeignetes e darstellbar. Also muss man sich fragen, wie viele solche $\boldsymbol{y}$ gibt es. Im Rahmen des RSA wird das noch genauer betrachtet. Sicher sind es weniger als 33, dann z.B. y = 0 muss ausgeschlossen werden. Die Grösse von mod N macht es aus, es trotzdem sehr viele solcher Werte y hat.

Weitere Beispiele von Einwegpermutationen

Wir betrachten $(\mathbb{Z}_{17}^*, \odot_p)$ und berechnen alle Potenzen $y = 3^x \; mod \; 17$.

$x \in \mathbb{Z}_{17}^*$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$3^x \; mod \; 17$	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

Nun sehen wir, dass die Elemente von $\mathbb{Z}_{17}^*$ neu angeordnet werden. Theoretisch gäbe es $16! \approx 2{,}1 \cdot 10^{13}$, solcher Anordnungen. Mit der Formel $y = g^x \; mod \; 17$, g ist ein Generator $(\mathbb{Z}_p^*, \odot_p)$ gibt es nur $\varphi\left(\underbrace{\varphi(17)}_{=16}\right) = \varphi(2^4) = (2-1) \cdot (2^{4-1}) = 2^3 = 8$

Generatoren und somit auch nur 8 solcher Anordnungen.

Wir betrachten nun die Kurve $E: y^2 \equiv x^3 + x + 6 \; mod \; 11$ und den Punkt P(2; 4). Ohne auf Details einzugehen, gebe ich an, dass die Kurve genau 13 Punkte enthält und dass der Punkt P(2; 4) mit k*P all diese Punkte erzeugt.

k	1	2	3	4	5	6	7	8
k*P	(2; 4)	(5; 9)	(8; 8)	(10; 9)	(3; 5)	(7; 2)	(7; 9)	(3; 6)

k	9	10	11	12	13	14	15	16
k*P	(10; 2)	(8; 3)	(5; 2)	(2; 7)	$\mathcal{O}$	(2; 4)	(5; 9)	(8; 8)

Wenn ich nun einen beliebigen anderen Punkt Q der Kurve – ausser der Nullpunkt $\mathcal{O}$ – nehme, so gibt eine andere Anordnung der Punkt k*Q. Es gibt somit 12 verschiedene Anordnungen. Theoretisch gäbe es aber $12! \approx 479'001'600$, denn der 13-te Punkt muss in diesem Falle immer $\mathcal{O}$ sein.

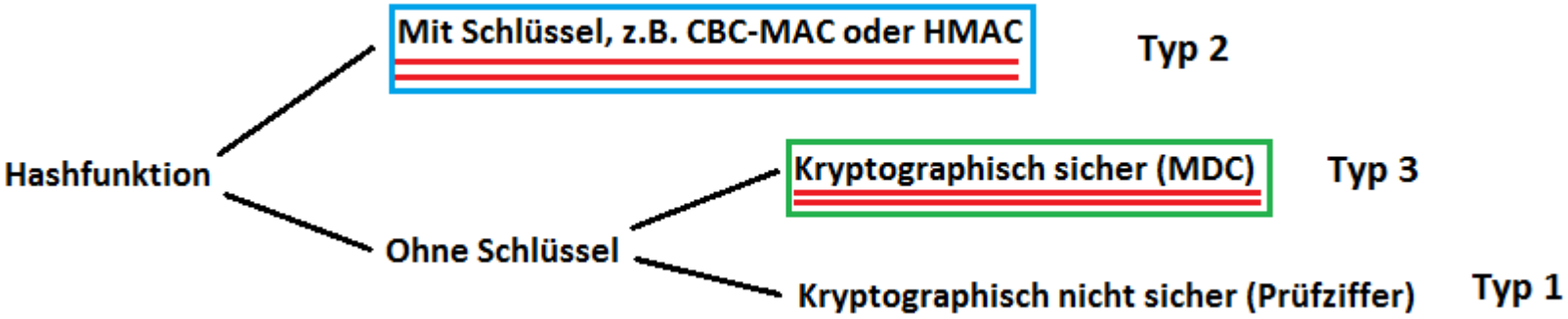
$y = g^x \bmod p$ als Einwegpermutation

In der Praxis (siehe Kap. 13.3 Diffie-Hellman und 16.1 Einleitung zu den Kennzahlen) ist muss die Primzahl p eine Grösse von 2000 (bis 2022) und danach 3000 Bit haben. Wobei man in einer Untergruppe der Ordnung q arbeitet, dabei ist q eine Primzahl der Grösse von 250 Bit (man nimmt ja dann 256 Bit). Da die Gruppenordnung prim ist, sind alle $\varphi(q) = q - 1$ Elemente Generatoren. D.h. es gibt dann $2^{256} - 1 \approx 2^{256} \approx 10^{77}$ möglicher Anordnungen. Das entspricht in etwa der Anzahl Atome im Weltall!!

Date	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash	
2020 - 2022	128	2000	250	2000	250	SHA-256 SHA-512/256 SHA-384 SHA-512	SHA3-256 SHA3-384 SHA3-512
2023 - 2026	128	3000	250	3000	250	SHA-256 SHA-512/256 SHA-384 SHA-512	SHA3-256 SHA3-384 SHA3-512

LÖSUNGEN DER AUFGABEN

Aufgabe 1



Aufgabe 2

- Beim Vertippen bei einer Zahl mit Gewicht 1, ist klar, dass es eine andere Summe mod 10 gibt.
- Beim Vertippen bei einer Zahl mit Gewicht 3, gibt es ebenfalls es eine andere Summe mod 10. Das liegt daran, dass 3*x mod 10 für x = 0, …, 9 immer einen anderen Wert gibt.

Berechnung 3x mod 10

x	0	1	2	3	4	5	6	7	8	9
3x mod 10	0	3	6	9	2	5	8	1	4	7

Beim Vertauschen ist zu zeigen, wann gilt x + 3y mod 10 = y + 3x mod 10

x	0	1	2	3	4	5	6	7	8	9
y	0	1	2	3	4	5	6	7	8	9
x+3y mod 10	0	4	8	2	6	0	4	8	2	6
3x+y mod 10	0	4	8	2	6	0	4	8	2	6
Also	0 ↔ 5	1 ↔ 6	2 ↔ 7	3 ↔ 8	4 ↔ 9					

Aufgabe 3

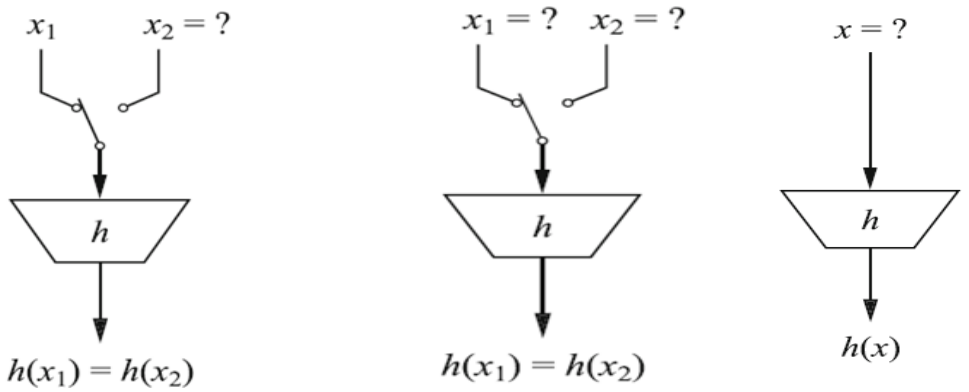
Hashfunktion	Kryptographisch sicher		Kryptographisch nicht sicher
	Mit Schlüssel	Ohne Schlüssel	
1) Alph. Registratur			X
2) Ablage nach Eingangsdatum			X
3) Parity-Bit			X
4) Byte Addition (mod 256)			X
5) CRC			X
6) CBC-MAC	X		
7) HMAC	X		
8) Konv. Prüfsumme			X
9) MDC = „Hashfunktion“		X	

Aufgabe 4 In der Nomenklatur der Einleitung um den Typ 3 (krypt. sichere Hashfkt. ohne Schlüssel

Aufgabe 5

a) Siehe rechts stehendes Diagramm.

b) Eine Prüfziffer (Parity-Bit, CRC-16, Prüfziffer am Schluss der ISBN-Nummer usw.) sind kryptographisch nicht sichere Hashfunktionen, da keine der 3 Sicherheitseigenschaften erfüllt ist.



Schwache Kollisionsresistenz (= zweite Urbildresistenz = 2nd-Pre-Image Eigenschaft)	**Starke Kollisionsresistenz** (oder Kollisionsresistenz)	**Urbildresistenz** (oder Einwegeigenschaft)

Aufgabe 6 Ein paar Zahlen für n = 384

Sie dürfen auch das Maple File in Ilias verwenden!

Es gibt total 2^n mögliche Hashwerte: $n = 384 \Rightarrow 2^n = 3{,}9 \cdot 10^{115}$

- Anzahl genau die Hälfte geändert: $\binom{384}{192} = 1{,}6 \cdot 10^{114}$, das entspricht ca. 4% aller möglichen Hashwerten. D.h. in ca. 4% aller Fälle wird genau die Hälfte = 192 aller Bits geändert, resp. nicht geändert.
- Anzahl der geänderten Bits zw. 182 – 202: $\binom{384}{182} + \cdots + \binom{384}{202} = 2{,}8 \cdot 10^{115}$, das entspricht ca. 71,6% aller möglichen Hashwerten. D.h. in ca. 71,6% aller Fälle werden zwischen 182 – 202 Bits geändert, resp. 202 – 182 nicht geändert.
- Anzahl der geänderten Bits zw. 162 – 222 → ca. 99,8%.
- Anzahl der geänderten Bits zw. 142 – 242 → ca. 99,999978%.
- Anzahl der geänderten Bits < 100 oder > 284 → ca. $5{,}5 \cdot 10^{-20}$%.
- Anzahl der geänderten Bits < 50 oder > 334 → ca. 10^{-51}%.

Aufgabe 7 Zunächst gibt es die folgenden Fakts, die zu erwähnen sind:

1) Die Sicherheitseigenschaft 1), also die Urbildresistenz, ist problemlos erfüllt.
2) Der CBC-MAC (u.ä.) sowie der HMAC sind Hashfunktionen mit Schlüssel.
3) Beim CBC-MAC entspricht die Hashgrösse (also der MAC) der Blockgrösse des zugrunde gelegten Blockchiffrieres, also z.B. 128 Bit bei AES. Beim HMAC beträgt die Grösse wegen dem „truncate to 96 bits“ eben nur 96 Bit.
4) Aufgrund der Grösse n = 128, resp. n = 96 sind die Sicherheitseigenschaften 2) & 3) so in diesem Sinne nicht erfüllt.
5) Nun wichtig ist, dass kryptographische Hashfunktionen mit und ohne Schlüssel, grundsätzlich nicht miteinander verglichen werden können. Die angegebenen Wahrscheinlichkeiten sind zwar gleich, aber man kann nicht mehr einfach so die zwei Angriffe durchführen. Zumindest müssten für den MAC und HMAC der Zugriff auf das Kryptomodul mit dem Schlüssel gewährleistet sein. Aber auch dann könnten i.d.R. (beschränkte Leitungskapazität zum Kryptomodul, keine Parallelisierung der Berechnungen möglich) nicht im selben Masse Meldungsvarianten durchsucht werden.

Resumée: Die Sicherheitseigenschaften 2) & 3) sind in diesem Sinne auch bei MAC und HMAC erfüllt, obwohl $n \geq 256$ bei Weitem nicht erfüllt ist.

Historische Notiz: Tatsächlich hat man zu Beginn vorgeschlagen einfach einen CBC-MAC (mit DES) mit fixem und publizierten Schlüssel als schlüssellose Hashfunktion zu verwenden. Dies wurde aber wegen der geringen Hashgrösse (n = 64) sehr schnell verworfen. Es war sehr schnell klar, dass n = 64 eine viel zu kleine Hashgrösse war.

Aufgabe 8 Wir gehen vom Gedankenspiel mit n zufällig angetroffenen Personen aus:

$$p = 1 - \overline{p} = 1 - \left(\frac{364}{365}\right)^n > \frac{1}{2} \Rightarrow -\left(\frac{364}{365}\right)^n > -\frac{1}{2} \Rightarrow \left(\frac{364}{365}\right)^n < \frac{1}{2} \Rightarrow n \cdot log\frac{364}{365} < log\left(\frac{1}{2}\right)$$

$$\Rightarrow n \cdot log\left(\frac{364}{365}\right) < log\left(\frac{1}{2}\right) \Rightarrow n \underbrace{\geq}_{(*)} \frac{log\left(\frac{1}{2}\right)}{log\left(\frac{364}{365}\right)} = 252{,}6$$

(*) Da $log\left(\frac{364}{365}\right) < 0$, ändert das Ungleichheitszeichen.

Aufgabe 9 In der Nomenklatur der Einleitung ist ein CBC-MAC vom Typ 2.

Aufgabe 10
Der neue Wert in $\boldsymbol{M_1}$ lautet nun: 21122024

Nachdem Eve den Block $\boldsymbol{M_1}$ abgeändert hat, muss sie noch den $\boldsymbol{IV}$ anpassen.

Dazu braucht sie die folgende Grundgleichung:

$$IVneu \oplus Block1neu = IV \oplus Block1 \Rightarrow IVneu = Block1neu \oplus IV \oplus Block1$$

Oder: $$?\oplus Block1neu = IV \oplus Block1 = Block1neu \oplus ? \Rightarrow ? = Block1neu \oplus IV \oplus Block1$$

Sie ändert im Datum die zweite Ziffer von 5 auf 1, somit gilt an dieser Stelle

$$IVneu = Block1neu \oplus IV \oplus Block1 = 1 \oplus B \oplus 5 = A \oplus 5 = F$$

Somit lautet der neue IV, der mitgeschickt wird: … **AFCD9876**…

Aufgabe 11 $m = MAC_{k_1,k_2}(x) = h(k_1 \parallel x \parallel k_2)$

Aufgabe 12 $m = HMAC_k(x) = h\big(k_O \parallel h(k_I \parallel x)\big) = h\big(k \oplus opad \parallel h(k \oplus ipad \parallel x)\big)$

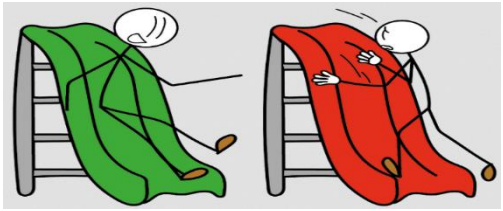
Aufgabe 13

a) Non rep. of receipt und Masquerade (sich für jemanden anders ausgeben).

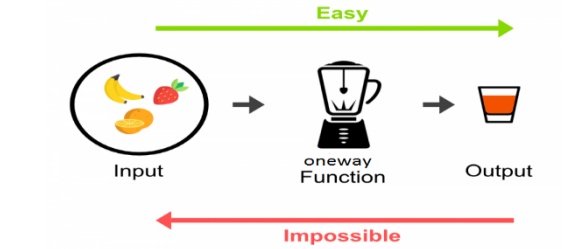
b) Masquerade (sich für jemanden anders ausgeben).

Aufgabe 14 Es ist eine Funktion von einer beliebig grossen Bitmenge in eine kleine (= Definition Hashfunktion). Die Urbildresistenz (= Einwegeigenschaft) ist nicht erfüllt.

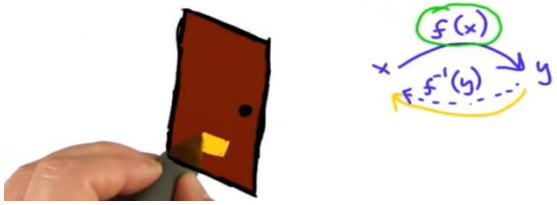
Aufgabe 15



Einwegfunktion mit Trapdoor (z.B. Klebemittel an den Schuhen)



Einwegfunktion ohne Trapdoor



Einwegfunktion mit Trapdoor, z.B. beim RSA ist es die Kenntnis der Faktorisierung von $N = p \cdot q$, damit kann $\varphi(N)$ und damit $d = e^{-1} \, mod \, \varphi(N)$ berechnet werden.

Aufgabe 16

NR	Aufgabe	Auswahl
a)	Welche Berechnungen sind im Rahmen von Einwegfunktionen von der Art „f easy to compute“?	☐ Die Ver- resp. Entschlüsselung mit XOR. ☐ Die Ver- resp. Entschlüsselung mit einer Blockchiffre. ☒ Die Berechnung der Werte, die beim DH Schlüsselaustausch über die Leitung geschickt werden. ☐ Die Berechnung des sogenannten diskreten Logarithmus beim Diffie-Hellman Schlüsselaustausches. ☒ Das Berechnen von $c = m^e \ mod \ N$ beim RSA. ☐ Das Berechnen von $m = \sqrt[e]{c} \ mod \ N$ beim RSA. ☒ Die Multiplikation $r \cdot P$ bei den Elliptischen Kurven. ☐ Das Bestimmen vom unbekannten k bei Elliptischen Kurven, falls $Q = k \cdot P$. ☐ Keine der Angaben ist zutreffend.
b)	Welche Berechnungen sind im Rahmen von Einwegfunktionen von der Art „f^{-1} hard to compute“?	☐ Die Ver- resp. Entschlüsselung mit XOR. ☐ Die Ver- resp. Entschlüsselung mit einer Blockchiffre. ☐ Die Berechnung der Werte, die beim DH Schlüsselaustausch über die Leitung geschickt werden. ☒ Die Berechnung des sogenannten diskreten Logarithmus beim Diffie-Hellman Schlüsselaustausches. ☐ Das Berechnen von $c = m^e \ mod \ N$ beim RSA. ☒ Das Berechnen von $m = \sqrt[e]{c} \ mod \ N$ beim RSA. ☐ Die Multiplikation $r \cdot P$ bei den Elliptischen Kurven. ☒ Das Bestimmen vom unbekannten k bei Elliptischen Kurven, falls $Q = k \cdot P$. ☐ Keine der Angaben ist zutreffend.
c)	Welche Berechnungen sind im Rahmen von Einwegfunktionen von der Art „f^{-1} with trapdoor information easy to compute“?	☐ Die Ver- resp. Entschlüsselung mit XOR. ☐ Die Ver- resp. Entschlüsselung mit einer Blockchiffre. ☐ Die Berechnung der Werte, die beim DH Schlüsselaustausch über die Leitung geschickt werden. ☐ Die Berechnung des sogenannten diskreten Logarithmus beim Diffie-Hellman Schlüsselaustausches. ☐ Das Berechnen der Potenzen beim RSA. ☒ Das Berechnen des ursprünglichen Wertes beim RSA. ☐ Die Multiplikation $r \cdot P$ bei den Elliptischen Kurven. ☐ Das Bestimmen vom unbekannten k bei Elliptischen Kurven, falls $Q = k \cdot P$. ☐ Keine der Angaben ist zutreffend.

Basis-Test Präsenz 4

Aussage	Richtig o. falsch?	Begründung
Eine kryptographisch sichere Hashfunktion ist auch eine Einwegfunktion.	☑ richtig ☐ falsch	
Eine Einwegfunktion ist auch eine kryptographisch sichere Hashfunktion.	☐ richtig ☑ falsch	Z.B. Die asymmetrische Kryptographie basiert auf solchen Einwegfunktionen, die keine Hashfunktionen sind.
Eine Prüfziffer schützt z.B. vor versehentlichem Verschreiben.	☑ richtig ☐ falsch	
Für SHA-3 kann man vier zentrale Sicherheitseigenschaften aufzählen.	☑ richtig ☐ falsch	Es sind dies die 3 zentralen Sicherheitseigenschaften. Die 4-te Eigenschaft ist, dass beim Ändern eines Bits im Input, im stat. Mittel jedes Bit mit Wahrscheinlichkeit ½ geändert wird.
Eine Hashfunktion (= MDC) muss seit 2023 mindestens 240 Bit Output haben.	☑ richtig ☐ falsch	
Den SHA3-224 darf noch lange für Signaturen verwendet werden.	☐ richtig ☑ falsch	Siehe vorherige Frage.
Die wichtigste Funktion eines Hashes ist die Komprimierung einer Meldung.	☐ richtig ☑ falsch	Es wird nichts komprimiert. Die wichtigste Funktion ist die Stellvertreterfunktion beim Signieren.
Der Kollisionsangriff ist weniger aufwendig, aber auch weniger realistisch als ein Pre-Image Angriff.	☑ richtig ☐ falsch	
Der Pre-Image Angriff ist weniger aufwendig, aber auch weniger realistisch als ein Kollisionsangriff.	☐ richtig ☑ falsch	
Den CBC-MAC gibt es in verschiedenen Ausprägungen.	☑ richtig ☐ falsch	

Basis-Test Präsenz 4, Fortsetzung

Aussage	Richtig oder falsch?	Begründung
Ein HMAC und ein CBC-MAC haben unterschiedliche Anwendungszwecke.	☐ richtig ☑ falsch	
Mit Blockchiffren kann man Zufallszahlen erzeugen.	☑ richtig ☐ falsch	
Mit Hashfunktionen kann man keine Zufallszahlen erzeugen.	☐ richtig ☑ falsch	
Die asymmetrische Kryptographie basiert vornehmlich auf Hashfunktionen.	☐ richtig ☑ falsch	
Die asymmetrische Kryptographie basiert vornehmlich auf Einwegfunktionen.	☑ richtig ☐ falsch	
Einwegfunktion mit Trapdoor ist ein Widerspruch in sich.	☐ richtig ☑ falsch	
Das Ziehen einer e-ten Wurzel mod N ($N = p \cdot q$ mit p, q grosse Primzahlen) ist eine Einwegfunktion ohne Trapdoor.	☐ richtig ☑ falsch	
Der diskrete Logarithmus ist eine Einwegfunktion ohne Trapdoor.	☑ richtig ☐ falsch	
Sei g ein Generator in $\mathbb{Z}_p^*$ und p eine Primzahl, dann ist $y \equiv g^x \ mod \ p$ eine Einwegpermutation.	☑ richtig ☐ falsch	Ein Generator g erzeugt per Definition alle Werte.

Danksagung

- Einige Folien entstammen aus der Vorlesung „Sichere Netzwerkkommunikation“ von Prof. Dr. A. Steffen, Hochschule Rapperswil. An dieser Stelle ein recht herzliches Danke schön an Hr. Andreas Steffen.
- Prof. Dr. Ueli Maurer, ETHZ, gewisse Folien entstammen aus seinem Modul „Information Security and Cryptography Fundamentals and Application“, ETH Zürich